

Assessment Cover Sheet

Programme Name:	BSc (Hons) Computing
Partner University Name:	Elizabeth School of London
Semester 1 or 2:	2
Module Code:	CMP600
Name of Lecturer:	Oti Edema
Due Date:	22 May 2026
Assessment Type:	Dissertation

<u>Declaration of Authenticity</u> (please check and sign):	☒
By submitting this assignment, I declare that this work is fully my own, and the work of others (including internet sources) is acknowledged by quotations and appropriate citing and referencing.	☒
I declare that this work has not made use of the work of any other student(s) past or present at this or any other educational institution or source.	☒
I declare that I have not commissioned another person to complete this work. This could include the use of professional essay-writing services, essay banks, or ghost-writing services.	☒
I declare that I have not asked another person to complete this work for me. This could include the help of friends, classmates, other students, or family members.	☒
I declare I have not used any AI software to help write this work outside of the assessment brief/guidelines (this includes translation tools, paraphrasing tools, or content creation websites such as 'ChatGPT'), other than for proofreading.	☒
I acknowledge that I have read all the texts listed in the bibliography or references list. I confirm all sources cited in the work appear in the bibliography or references list at the end.	☒
Sergiu Ionut Pascaru	

Areas for development identified at draft stages by feedback given by tutors	How I have addressed this feedback in this final submission
1. Ensure Harvard referencing is consistent throughout	Reviewed all in-text citations and bibliography entries for Harvard format compliance
2. Proofread for grammar and formatting consistency	Applied Arial 12pt double-spaced formatting throughout; proofread all chapters

3. Add web application screenshots to evidence appendix	Added 3 PhishGuard screenshots (Safe, Suspicious, Malicious verdicts) to Appendix F
I attended most support classes for this assessment:	YES
Number of support classes missed and why:	
For any missed support classes, I caught up and sought help if needed:	YES

Declaration of AI use in this academic work

It is important to acknowledge the use of AI tools in your work and to be clear about how you have used them. It is generally considered poor academic practice to include any content generated by GenAI tools in your work, unless you are explicitly reflecting on, analysing and evaluating the output of the software.

You should put an 'x' against any statement in the list below that you know is TRUE to honestly declare how you have used AI in your work:

Statements

- ☐ I can confirm that I have not used any AI tools within this piece of work.
- ☒ I have used AI tools to help me with developing ideas.
- ☒ I have used AI tools to help formulate a plan or structure for my assessment.
- ☒ I have used AI tools to assist me with my research/gathering information.
- ☒ I have used AI tools to help me understand key theories and concepts.
- ☒ I have used AI tools to identify themes as part of my data analysis for my work.
- ☐ I have used AI tools to give me feedback on a draft piece of work.
- ☒ I have used AI tools to proofread my work for correct spelling and grammar.
- ☒ I have used AI tools to generate tables, figures or diagrams.

A Machine Learning Approach to Real-Time Phishing URL Detection

Dissertation

**A Machine Learning Approach to Real-Time Phishing URL
Detection:
A Comparative Analysis of Classification Models**

Supervisor: , Oritsetimeyin Edema

BSc (Hons) Computing [Top-up]

Word Count: 6,996

by

Sergiu Ionut Pascaru

Registration No.: 2310-111729

Academic Year 2025–2026

Elizabeth School of London

*Copyright Elizabeth School of London. No part of this publication may be reproduced without the prior
written permission of the copyright owner.*

Abstract

Phishing is one of the most pervasive and damaging cyber threats, that takes advantage of the vulnerability of humans and not of systems to obtain sensitive information. The traditional way of detecting malicious domains (blacklists) is reactive and cannot keep up with the rapid creation of new domains. In this dissertation, the effectiveness of machine learning models trained solely with lexical features, which are structural and statistical features that can be directly derived from the URL string, will be studied for phishing URL detection. This is done without having to query outside sources like DNS or analyze the content on the page, which is intended to be a quick, “first line of defence” on the client side.

This study is based on the methodology of Cross-Industry Standard Process for Data Mining (CRISP-DM). The data set consisted of 641,053 URLs which were cleaned and processed to remove ten lexical features, such as URL length, domain length, path length and character counts (Siddhartha, 2024). Logistic Regression (baseline linear), Random Forest (non-linear ensemble), and Support Vector Machine (LinearSVC) classifiers were tested.

The results show that the accuracy, F1-Score and ROC AUC of the Random Forest model is significantly higher than the linear models, at 94.03%, 0.9120 and 0.9855, respectively. Most importantly for deployment in the real world, it had the lowest False Positive Rate (FPR) of 5.45%, which reduces the number of legitimate URLs that are incorrectly blocked. Cross-validation showed the stability of these results and ANOVA feature selection revealed that dot count,

path length and slash count were the most discriminative features. The results suggest that the relationship of the lexical features of the URL and malicious intent is highly non-linear and that ensemble methods such as Random Forest perform very well for real-time, low latency phishing detection.

Dissertation Declaration

Statement of Originality and Authenticity

I confirm that the dissertation I am submitting is an original and authentic piece of work written by myself that satisfies the Elizabeth School of London rules and regulations with respect to Plagiarism and Collusion. I further confirm that I have fully referenced and acknowledged all material incorporated as secondary resources in accordance with the Harvard system.

I also certify that I have taken a copy of the dissertation, which I will retain until after the Board of Examiners has published the results, and which I will make available on request in pursuance of any appropriate aspect of the marking and moderation of the work within the University Regulations.

Name: Sergiu Ionut Pascaru

Registration Number: 2310-111729

Programme: BSc (Hons) Computing [Top-up]

Study Centre: Elizabeth School of London

Date: 22 May 2026

Please note that Dissertations will not be assessed without the inclusion of this declaration by the student.

Please note that Dissertations may not be assessed without the inclusion of electronic copies which may be submitted through Turnitin software to check for authenticity.

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Oritsetimeyin Edema, for their invaluable guidance, continuous support, and constructive feedback throughout this research project. Their expertise and encouragement have been instrumental in shaping this dissertation and keeping the project on track through each sprint.

I also wish to thank the academic staff at the Elizabeth School of London and Newcastle College Group for providing a stimulating learning environment and the resources necessary to complete this work. The module leader, Ashfaque E. Alam, provided essential guidance during the proposal stage that helped define the research direction.

Finally, I am deeply grateful to my family and friends for their unwavering support, patience, and understanding during the demanding months of this project. Their encouragement made the long hours of coding, testing, and writing far more manageable.

List of Figures and Tables

Figures

Figure 1: CRISP-DM Methodology Framework

Figure 2: Project Gantt Chart (Sprint Schedule)

Figure 3: System Architecture Diagram

Figure 4: Data Pipeline Flowchart

Figure 5: Binary Label Distribution After Cleaning

Figure 6: URL Length Distribution by Label

Figure 7: Logistic Regression Confusion Matrix

Figure 8: Random Forest Confusion Matrix

Figure 9: SVM (LinearSVC) Confusion Matrix

Figure 10: Combined ROC Curves for All Models

Figure 11: Random Forest Feature Importance

Figure 12: ANOVA F-Score Feature Ranking

Figure 13: Cross-Validation Performance Heatmap

Tables

Table 1: Functional Requirements

Table 2: Non-Functional Requirements

Table 3: Risk Assessment Matrix

Table 4: CRISP-DM to Sprint Mapping

Table 5: Tools and Technologies

Table 6: Lexical Features Engineered

Table 7: Final Model Performance Comparison

Table 8: 5-Fold Cross-Validation Results

Table 9: ANOVA F-Score Feature Ranking

List of Abbreviations

Abbreviation	Full Term
APWG	Anti-Phishing Working Group
AUC	Area Under the Curve
CRISP-DM	Cross-Industry Standard Process for Data Mining
DNS	Domain Name System
EDA	Exploratory Data Analysis
FN	False Negative
FP	False Positive
FPR	False Positive Rate
IC3	Internet Crime Complaint Center
LR	Logistic Regression
ML	Machine Learning
NLP	Natural Language Processing
RF	Random Forest
ROC	Receiver Operating Characteristic
SVM	Support Vector Machine
TN	True Negative
TP	True Positive
URL	Uniform Resource Locator
WHOIS	Who Is (Domain Registration Query Protocol)

Table of Contents

Assessment Cover Sheet	1
Dissertation Declaration	7
Statement of Originality and Authenticity	7
Acknowledgements	8
Figures	9
Tables.....	9
List of Abbreviations	11
Table of Contents.....	12
Chapter 1: Introduction.....	15
1.1 Background to Study.....	15
1.2 Problem Statement	16
1.3 Research Aim and Questions	17
1.4 Objectives of the Study	18
1.5 Significance of the Study	19
1.6 Structure of the Dissertation.....	19
Chapter 2: Literature Review	21
2.1 Introduction to Literature	21
2.2 Theoretical Framework	21
2.3 Key Themes and Findings	22
2.3.1 System Performance: Blacklisting vs. Machine Learning.....	22
2.3.2 Security Issues: Feature Categories and Trade-offs.....	23
2.3.3 Algorithm Efficiency: Linear vs. Non-Linear Models.....	23
2.3.4 Dataset Scale and Reproducibility in Phishing Research.....	24
2.3.5 User Experience: The False Positive Rate Challenge	25
2.3.6 The Evolution of Phishing Tactics.....	26
2.4 Research Gaps.....	27

2.5 Summary	27
Chapter 3: Research Methodology	29
3.1 Research Philosophy	29
3.2 Research Approach	29
3.3 Research Choice	30
3.4 Research Strategy	30
3.5 Research Design	31
3.6 Data Collection Methods	32
3.7 Population and Sampling	32
3.8 Data Analysis Techniques	33
3.9 Ethical Considerations	33
3.10 Limitations of the Methodology	34
Chapter 4: System Design and Implementation.....	35
4.1 System Overview	35
4.2 System Architecture.....	35
4.3 Technology Stack	36
4.4 System Design.....	36
4.5 Implementation Process	37
4.5.1 Data Cleaning and Preprocessing (Sprint 3).....	38
4.5.3 Model Training (Sprints 5, 6, and 7).....	40
4.5.4 Hyperparameter Selection Rationale	41
4.6 System Testing	42
Chapter 5: System Evaluation, Results, and Discussion	43
5.1 Introduction.....	43
5.2 Testing Environment.....	43
5.3 Performance Evaluation.....	43
5.4 Cross-Validation Stability	44

5.5 Feature Selection and Importance	45
5.6 Limitations of Results.....	46
5.7 Discussion: Interpretation of Results.....	46
5.8 Discussion: Comparison to Literature	48
5.9 Discussion: Implications for Practice.....	49
5.10 Discussion: Strengths of the Study	49
Chapter 6: Conclusion.....	51
6.1 Summary of Key Findings.....	51
6.2 Research Contributions	51
6.3 Implications for Business Practice.....	52
6.4 Limitations of the Study.....	52
6.5 Recommendations for Future Research.....	53
6.6 Personal Reflection.....	54
Bibliography	56
Appendices	58
Appendix 1: Ethics Checklist.....	58
Appendix 2: Supervisor Meeting Minutes.....	58
Appendix 3: Risk Assessment	59
Appendix 4: Sprint Schedule (Gantt Chart)	59
Appendix 5: Reflective Career Skill Development Summary	61
Appendix 6: Additional Materials.....	62
Appendix A: Data Exploration and Preprocessing Evidence.....	63
Appendix B: Sprint 5 — Logistic Regression Baseline Results	69
Appendix C: Sprint 6 — Random Forest Results.....	72
Appendix D: Sprint 7 — Support Vector Machine Results	75
Appendix E: Sprints 8-9 — Cross-Validation, Feature Selection, and Final Comparison.....	78
Appendix F: PhishGuard Web Application Prototype.....	83

Chapter 1: Introduction

1.1 Background to Study

The Internet is an integral part of the world communication, commerce and social interaction. However, this prevalence has also made it a big target for cybercriminals. Phishing is one of the most common and successful methods of attack among the many cyber threats. Phishing is a form of social engineering, where the attackers pose as a trusted entity to make the victim divulge credit card information, financial details, login information and other personal information (Sahingoz et al., 2019). According to the Anti-Phishing Working Group (APWG), the attacks on phishing have continued to grow in 2023, and over 4.7 million attacks were reported worldwide during the year, the most attacks ever reported (APWG, 2024).

Phishing attacks are not like many cyber-attacks that take advantage of software vulnerabilities, but rather, they take advantage of the psychology of humans. Attacks are frequently made by sending deceptive emails, messages or websites that pretend to be from a legitimate service or source. Most phishing attacks are based on the basic technique of the malicious Uniform Resource Locator (URL). The attacker, by enticing the user to click on a link, redirects him or her to a fake Web page that can be utilized to steal information or install malware. There is an economic impact – in 2022 the FBI's Internet Crime Complaint Center (IC3) received reports of over \$10.3 billion in losses to the public resulting from internet crime, with phishing being the most reported attack vector (FBI, 2023).

Blacklisting, also referred to as using a list of known bad URLs, has been the traditional way of detecting phishing. When the user visits a URL, the system can determine if the URL is blacklisted. Blacklists are reactive and are good against known threats. There is a “patient zero” issue – for them to be blocked, a malicious URL needs to be identified and be placed on the list. The attackers can create thousands of new ephemeral domains daily and blacklists are never up to date (Ma et al., 2009). Sheng et al. (2009) reported that it takes on average 12 hours after the reporting of a new phishing URL to be included into major blacklists, and often, most victims have already been compromised.

1.2 Problem Statement

It would be an understatement to say how big the phishing problem is. The FBI Internet Crime Complaint Center (IC3) estimates that phishing is the top category of cybercrime in 2022, and the losses in the U.S. alone were over \$52 million.

The global impact is much greater when considering unreported incidents, reputational damage to impersonated brands and downstream costs of credential theft that result in additional breaches. It is particularly affecting small and medium-sized businesses, whose companies don't have the security infrastructure of larger organizations.

The basic problem with current phishing detection systems is that of a latency vs. accuracy trade-off. Content-based methods, which process the HTML, JavaScript or visual structure of a web page, can be very accurate but are subject to a delay as the page loads, and the user is presented with the page while the analysis is taking place, which may lead to "drive by" downloads during

this period. Host-based methods that retrieve DNS records, WHOIS databases or server geolocation are also not good for real-time, client-side installation, like in a browser extension, and involve network latency.

An interesting alternative is to analyze the structural and statistical characteristics of the URL string, known as lexical analysis. Lexical features can be obtained from the URL string in real time, without making any request to the network, which makes them well suited for low latency, client-side protection. The key question, however, is whether lexical features are enough to provide accuracy and importantly, False Positive Rate (FPR), needed for a practical detection system.

1.3 Research Aim and Questions

The main goal of this dissertation is to create, implement and test a machine learning pipeline that classifies phishing URLs based on only the lexical features of the URL string.

The main research question for this study is:

Are machine learning models that are trained only on lexical features of URLs able to provide high accuracy and low False Positive Rates (FPR) that are practical for real-time, client-side phishing detection?

The following sub-questions were developed to operationalise this question.

Which of the three classifiers (Logistic Regression, Random Forest and SVM) has the highest F1-Score and True Positive Rate? Second, which model results in the smallest False Positive Rate which disrupts the activity of legitimate users

the least? Third, how is the relation between lexical features and malicious intent – linear or non-linear?

1.4 Objectives of the Study

The research aim is broken down and translated into a series of specific and measurable objectives that together answer the research question. Every objective will be broken down into a different phase of the CRISP-DM methodology and will be linked to a different sprint in the project timeline. The objectives were set with the aim of being attainable within the academic timeframe, and at the same time providing enough rigor to yield meaningful findings.

There are five measurable objectives to achieve the stated aim. Second, to download and preprocess Malicious URLs Dataset (Siddhartha, 2024) from Kaggle and do rigorous data cleaning, deduplication and binarization. Third, to create a fast and vectorized feature extraction engine to extract 10 meaningful lexical properties from raw URL strings. Third, to train and optimize three different machine learning classifiers that belong to three classes of different algorithmic paradigms: linear, non-linear ensemble and kernel-based classifiers. Fourth, to thoroughly test the models with a broad range of metrics with a particular emphasis on minimizing False Positive Rate. Fifth, to perform statistical feature selection to find the most indicative lexical features that can be used for phishing detection.

1.5 Significance of the Study

This study has several contributions to the field of cybersecurity. It offers an empirical and controlled comparison of linear and non-linear classifiers, but only with purely lexical features, which have not been compared in the literature, while other studies tend to combine different types of features. The emphasis on False Positive Rate as a major criterion is due to the real-life deployment scenario: If a system blocks legitimate sites, users will turn it off in a heartbeat. Also, the relatively small number of features allows the results to be easily transferred to other resource-limited applications like browser extensions or mobile apps.

Academically, the study provides a replication methodology, which other researchers can expand. All code, data and hyperparameter settings are freely available on GitHub, allowing for verification and extension. Practically, the study shows that one can detect phishing without having to rely on expensive infrastructure or complex feature engineering. A straightforward set of 10 structural features, along with a suitable ensemble algorithm, can be used to achieve a level of protection that is much greater than linear methods. This has immediate implications for developers creating security tools for end users that may not be able to have enterprise-level protection systems.

1.6 Structure of the Dissertation

The rest of this dissertation consists of the following. The theoretical background of phishing, current phishing detection techniques and previous research on lexical analysis is discussed in chapter 2. Chapter 3 provides information on the research methodology that is followed in the research such as CRISP-DM

framework, research philosophy, data collection and ethical considerations.

Chapter 4 covers the system design and implementation, data cleansing, feature engineering, and model training. The results of the evaluation are presented in chapter 5 where the performance of the models is compared by tables, charts and statistical tests. A critical discussion of the results, in the context of the research question and literature, is also included in Chapter 5. Chapter 6 provides a summary of the findings, highlights limitations and offers suggestions for further research.

Chapter 2: Literature Review

2.1 Introduction to Literature

This chapter aims to critically review the existing knowledge about phishing URL detection using machine learning. The review has been done thematically starting with the theory behind the URL analysis, then the important findings from literature and then the gaps found in literature are identified which this dissertation addresses. It was evident that the literature was found from relevant and peer reviewed journals, conference proceedings and authoritative industry report, based on the academic writing principles of Bailey (2018) and Swales and Feak (2012).

2.2 Theoretical Framework

To understand how lexical phishing detection works; you need to have a background understanding of how a Uniform Resource Locator (URL) works. The elements of a standard URL are scheme – the protocol to use; domain – the hostname of the resource; path – the location of the resource; query – a query to pass to the resource; fragment – an internal page reference within the resource. These elements are the target of manipulation by the attackers themselves using well-known methods. Typo squatting is when the domain names are registered that look like the legitimate ones, e.g. "g00gle.com" instead of "google.com". Subdomain stacking consists of using long subdomains to obscure the real malicious domain especially when viewed on mobile devices. A Deep Directory Structure is when path obfuscation is used to create deep and complex directory

structures, designed to obscure the nature of the page. These structural changes are the very same we want lexical feature analysis to discover.

The theoretical motivation for solutions to this problem based on machine learning is based on the fact that malicious URLs have statistically different structural properties than benign urls as the attackers have to obfuscate their identity or the fact that they must evade filters. This assumption has been empirically proved by several research studies, especially by Ma et al. (2009) that URL features alone are able to provide sufficient prediction power for classification.

2.3 Key Themes and Findings

2.3.1 System Performance: Blacklisting vs. Machine Learning

The oldest, and most used technique for detecting phishing is blacklisting, such as Google Safe Browsing. Blacklists, although very accurate in determining if a known threat exists and having very few false positives, are in essence reactive. Blacklists are known to not catch newly registered phishing sites until the first few hours of their life, according to the research done by Sheng et al. (2009). The "zero-day" gap, the period between the creation of a URL and when it gets added to a blacklist, is a very critical period of vulnerability. To overcome this, machine learning methods have been proposed, which learn patterns and not specific URLs, and classify previously unknown domains.

2.3.2 Security Issues: Feature Categories and Trade-offs

Phishing detection solutions can be classified into three categories, which have different security implications. Content-based features are based on the HTML Document Object Model, hidden frames or the visual layout of the webpage. These techniques are very effective but are costly in terms of computation and require the page to be loaded - this may be enough to let the user be exposed to drive-by downloads during the analysis window. Host Based features perform analysis of either DNS records, WHOIS information or server geolocation. These are vulnerable to attackers who can circumvent them through fast-flux DNS and/or corrupted legitimate servers and account for network latency. Lexical features (also known as URL features) examine the actual string of a URL and do not require any network call to retrieve the features, hence can be used in real time and client-side deployments.

2.3.3 Algorithm Efficiency: Linear vs. Non-Linear Models

Using machine learning techniques on lexical URL analysis has been a rich field of study. Sahingoz et al. (2019) performed extensive research on phishing detection from URLs using machine learning techniques, designing different lexical features such as word counts, character frequencies and NLP features. They have found that ensemble methods, especially Random Forest, always gave a better performance as compared to linear models and individual decision trees. Their feature set contained however features which are quite costly to compute from the NLP side, e.g., bag-of-words representations, which may not be suitable for a lightweight client-side tool.

Ensemble methods are theoretically justified in this area, as was done by Breiman (2001), who created Random Forests. Lexical features may have complicated and non-linear interactions. For instance, a long URL can't be inherently bad – there are lots of dynamic pages with long URLs, but a long URL with numerous dots and a particular keyword may be extremely suspicious. Many of these multi-dimensional interactions are hard for linear models (e.g., Logistic Regression or standard Support Vector Machines (Cortes and Vapnik, 1995)), to model without explicitly adding polynomial features. Random Forests model these non-linear boundaries by building several decision trees on a random selection of features and data.

2.3.4 Dataset Scale and Reproducibility in Phishing Research

One of the major problems with phishing detection research is that there are no large, representative and open data sets. Many of the initial studies have been based on small, commercial sets of URLs with less than 10,000 sample URLs and as such, the results of these studies cannot be generalized. Thanks in part to the development of public, large-scale datasets on sites like Kaggle, some of this concern has been alleviated, and researchers put models to the test on hundreds of thousands of labelled examples. But the time frame is still a challenge – phishing campaigns are short-lived: most malicious domains are only up for less than 48 hours before they are either shut down or blacklisted. Data gathered in one period might not be representative of the variety of strategies used by attackers. Even with such restrictions, static data sets are the most common data

used in comparative studies, as they can be used to control, and reproduce experimentation with different algorithms and feature sets.

Reproducibility is a major issue in the field. Datasets, feature extraction code, and specific hyperparameter settings are not published in many studies, so it is not possible to independently verify these studies. To ensure the project is reproducible, all the processing code has been posted on GitHub, all preprocessing decisions documented and fixed random seeds used. A large public dataset (more than 640,000 urls) makes it possible to obtain results with good statistical power and to exactly reproduce the experiments by other researchers.

2.3.5 User Experience: The False Positive Rate Challenge

One of the key aspects of detection systems that has not been adequately discussed in literature is user experience. Rao and Pais (2020) discussed Deep learning methods to classify URL based on character level embeddings that are given to convolutional neural network (CNN). Some of their findings on accuracy were positive but they did not do any specific analysis to look at False Positive Rate – the percentage of legitimate URLs which were flagged as malicious. If a high FPR was used in real deployment, it would be impossible to use the tool since the users would simply turn off a system that is blocking good sites. This literature gap serves as an impetus for the present study to highlight explicitly FPR as one of the main evaluation metrics.

2.3.6 The Evolution of Phishing Tactics

To understand the selection of phishing features, it is important to understand the changing tactics of phishing attacks. The early phishing attacks used some of the simplest ways of hiding the URL like IP address substitution and too many subdomains. Users were getting more aware of these patterns, attackers started using typo squatting – domain names that look like well-known brands, but with minor characters in the name changed. In more recent years, attackers have begun to use URL shortening services and compromise popular and well-known sites for the purpose of embedding the phishing content in them, making it more difficult to detect these using domain alone. This evolutionary trend reinforces the need for a combination of lexical characteristics—no one measure alone—to be used to detect attacks, as attackers will continuously make changes to work around simple detection rules.

According to the Anti-Phishing Working Group (APWG, 2023), phishing attacks peaked in 2022, and they have been detected across the globe to the tune of more than 4.7 million. One of the main points the report noted was that one of the major targets of attacks has been mobile users, who display shortened URLs in their browser, making it virtually impossible for an eyeball to check the URL. With this move, it further encourages the creation of automated and real-time detection systems which can analyse URLs even before they get rendered; whatever their context of display is.

2.4 Research Gaps

Although there are existing publications that prove the effectiveness of machine learning for lexical analysis, there are still a couple of gaps in this field. For one, many studies focus on overall Accuracy or AUC but not enough on the False Positive Rate which is the key measure of usability when deploying the system in the real world. Secondly, numerous NLP studies employ intricate lexical features that require a lot of memory and processing to be able to store all the words in large vocabularies. Thirdly, there are no controlled direct comparisons between linear and non-linear classifiers on the same purely structural feature set on a large-scale data set.

This dissertation aims to fill these gaps by restricting the attention to a small selection of purely structural and statistical lexical features – lengths, counts, ratios and entropy – for which computation is easily done. The evaluation specifically focuses on the FPR, to determine if this small set of features and a powerful ensemble modelling approach can be used to obtain a small FPR threshold needed for practical use.

2.5 Summary

Based on the literature review, the authors have concluded that detection of phishing has gone through three generations namely blacklisting, content/host based and lexical machine learning. For lexical classification, ensemble method has been shown to be the best method, in particular, Random Forest. But the explicit assessment of False Positive Rate (FPR) and the only usage of the

structural features in large-scale datasets are still under explored. The aim of this dissertation is to endeavor to fill some of these gaps by comparing two experimental conditions in a controlled experiment.

Chapter 3: Research Methodology

3.1 Research Philosophy

This research is based on the positivist philosophy which believes that knowledge is based on observable, measurable phenomena and objective truths can be fixed through systematic experimentation (Saunders, Lewis and Thornhill, 2019). This study adopts a positivistic approach in its research because its research question is basically empirical, which is to determine if a machine learning model can be designed that has certain measurable performances. This is done by using objective metrics (Accuracy, F1-Score, FPR, ROC AUC) that are not dependent on the interpretation of the researcher, and which can be replicated and are objective.

3.2 Research Approach

The method employed is deductive approach that is from general to specific. Literature indicates that the multi-dimensional interconnections between the features are complex and thus one can expect that the non-linear ensemble method will give better performance on the lexical features of the URLs. Controlled experiments are used to support this hypothesis and experimental results fit with or fail to fit with, theoretical prediction. Deductive approach ensures the research is theory based and research is contributing to the theory.

This research question was testable, specific, therefore the approach used was deductive approach. More suitable in exploratory qualitative research would be an inductive approach of collecting data without having a hypothesis first and

letting the pattern develop out of this. What the literature has already suggested here is a good collection of theories that superiority will be non-linear; the point of this research is not to come up with a new theory from scratch, but to do a rigorous test of the 'theory of non-linear superiority'.

3.3 Research Choice

In this study, the research types used are research types with pure qualitative approach. The research question needs the objective comparison of the classifiers and can only be achieved with quantitative methods. Qualitative approaches (interviews etc., and thematic analysis etc.) are not suitable because there are no human subjects, and no subjective experiences in it. The quantitative approach enables the researcher to be statistical, replicable, and have generalizable results.

3.4 Research Strategy

Experimental approach to the study is used. Finally, three classifiers, all based on machine learning methods are trained with the same features, data set, and train and test split, making for a fair and direct comparison. Classification algorithms (Logistic Regression, Random Forest and SVM) are the independent variable. The performance metrics (Accuracy, Precision, Recall, F1-Score, FPR, ROC AUC) are the dependent variables. To control confounding variables, stratified splitting, balanced class weights and fixed random seeds for reproducibility are used.

3.5 Research Design

The type of research design in this study is explanation which seeks to explain the relationship between the URL features of the lexical and phishing classification accuracy. The design is based on the Cross-Industry Standard Process for Data Mining (CRISP-DM) framework (Chapman et al., 2000) that offers a structured and iterative manner for tackling data science projects. The CRISP-DM model is a six-stage model which includes the following: Business Understanding, Data Understanding, Data Preparation, Modelling, Evaluation and Deployment. The relationship of these phases with the work sprints of the project in accordance with Agile methodology is detailed in Table 4.

Table 4: CRISP-DM to Sprint Mapping

CRISP-DM Phase	Sprint(s)	Activities
Business Understanding	Sprint 1	Project setup, tools selection, proposal writing
Data Understanding	Sprint 2	Exploratory Data Analysis, class distribution, initial visualisations
Data Preparation	Sprints 3–4	Cleaning, deduplication, binarisation, lexical feature engineering
Modelling	Sprints 5–7	Logistic Regression, Random Forest, SVM training
Evaluation	Sprints 8–9	Cross-validation, ANOVA feature selection, final comparison
Deployment	Sprints 10–14	Dissertation write-up, GitHub repository finalisation, submission

3.6 Data Collection Methods

This study's data was obtained from the Malicious URLs Dataset, which was published by Siddhartha (2024) on Kaggle. There was no primary data collection from human subjects, rather this was a secondary data source. The reasons for choosing the dataset are it has more than 650,000 records, providing sufficient data to create a good model; there is a diversity of benign and malicious URL types in it, which ensures diversity in the data; it is already labelled, which means it does not require manual annotation.

The original data set comprised of four classes: benign, phishing, defacement and malware. These were then mapped on to binary target (benign-0, malicious-1) to conform them for binary classification. This binarization is a consequence of the use case that we have in mind in real world, which is that the system should be able to make a safe/unsafe decision.

3.7 Population and Sampling

The population for this study is the entire population of URL data records contained in the Kaggle data set (651,191 data records). The sample was de-duplicated and cleaned up yielding 641,053 unique URLs. Using stratified sampling the class distribution was maintained for the training set (80% of data, 512,842 rows) and held out test set (20% of data, 128,211 rows). A subsample of 100,000 rows (50,000 for each class) was chosen for cross validation for this reason of representativeness and practicability of computation.

3.8 Data Analysis Techniques

Various evaluation measures were computed from the Confusion Matrix which were used to assess the model performance. Accuracy – Overall level of correctness of performance. Precision: How many predictions were right / Total number of predictions that were positive. Recall (Sensitivity) is the percentage of the actual positives that are detected out of all the actual positive. Harmonic means of Precision and Recall is the F1-Score. There is one safety measure that is the most important usability measure: False Positive Rate ($FPR = FP / (FP + TN)$). ROC AUC is an overall measure of performance which summarizes the performance over all cut points. Further, features were ranked according to their discriminative ability by using feature selection method based on ANOVA F-test (SelectKBest with `f_classif`).

3.9 Ethical Considerations

This project received an ethics approval form (HE35 dated 09 December 2025) after going through the process of institutional ethical review. All the data is publicly available from Kaggle and has no personally identifiable information. No humans were involved in the study. All the malicious URLs were treated like plain text strings, and never evaluated, resolved and rendered in browser. This analysis was performed 100% offline and in a secure Python environment to not make any inadvertent network connections to malicious sites. Throughout the NCG Research Ethics Policy and Procedures were adhered to.

The other ethical concern is when the research has a “dual use.” A model which identifies phishing URLs can be reverse engineered to produce phishing URLs

which will not be detected by the model. This risk was reduced by concentrating on the structural aspects which are inherent to the URL format and not on specific evasion techniques. The code published does not contain any code for generating URLs. In addition, the model is designed for defensive use and the public release of the model weighs in favor of the security of other Internet users, not its malicious use.

3.10 Limitations of the Methodology

A few methodological limitations are pointed out. Firstly, this is a single snapshot in history, and it may not be representative of the evolution of attackers over the years. Second, because of the restriction to lexical features, it will not be able to detect phishing URLs with the same structure as the benign URLs. Thirdly, only three classification algorithms were compared, the gradient boosting algorithms such as XGBoost or deep learning algorithms were not used due to focus and computational feasibility. Fourth, all computations will need a balanced subsample for the cross validation, and this may not reflect natural class distribution. These restrictions, however, do not affect the effort to maximize the internal validity of the methodology by carefully controlling the experimental conditions, the evaluation protocols and the random seed numbers, to make the comparative results obtained by different algorithms robust and reliable.

Chapter 4: System Design and Implementation

4.1 System Overview

The phishing URL detection system has been designed as a modular pipeline implemented in Python code to be easily reproducible, maintainable and to ensure separation of concerns. The pipeline can be fed with raw URL strings, and will return 10 lexical features, and a binary classification (benign or malicious) with a probability score. All the code is available in GitHub (<https://github.com/s3giu/phishing-url-detection>) and is in development using Agile sprints on Trello (<https://trello.com/b/YbG9606p/phishing>).

4.2 System Architecture

The system consists of three stages with pipeline architecture. The first step is called Preprocessing (provided by `src/preprocessing/`) and involves data cleaning, deduplication, binarization and URL tokenisation. In the second stage Feature Engineering, the ten lexical features are extracted from each URL and are vectorized, by using panda's operations in the `src/features/` package. Model Training and Evaluation is the third stage, which is performed by `src/models/`, and it trains the classifiers, makes predictions and outputs evaluation artefacts like confusion matrix, roc curve and summary statistics.

4.3 Technology Stack

Table 5: Tools and Technologies

Tool / Library	Version	Purpose
Python	3.11	Core programming language
pandas	2.x	Data manipulation and vectorised feature extraction
NumPy	1.x	Numerical computation
scikit-learn	1.x	ML models, metrics, cross-validation, feature selection
matplotlib / seaborn	3.x / 0.13	Data visualisation and evidence generation
Git / GitHub	Latest	Version control and repository hosting
Trello	Web	Agile project management and sprint tracking
VS Code	Latest	Integrated development environment

4.4 System Design

The system design was guided by the functional and non-functional requirements established during the Business Understanding phase.

Table 1: Functional Requirements

ID	Requirement	Priority
FR1	The system shall accept a raw URL string as input	Must Have
FR2	The system shall extract lexical features without external network queries	Must Have
FR3	The system shall classify the URL as benign (0) or malicious (1)	Must Have

FR4	The system shall output a probability score alongside the binary classification	Should Have
FR5	The system shall generate evaluation metrics including FPR, Accuracy, and ROC AUC	Must Have

Table 2: Non-Functional Requirements

ID	Requirement	Priority
NFR1	Feature extraction shall be completed in under 1 millisecond per URL	Must Have
NFR2	The model shall achieve a False Positive Rate below 10%	Must Have
NFR3	The pipeline shall be reproducible with a fixed random seed	Must Have
NFR4	The codebase shall be modular and version-controlled via GitHub	Should Have
NFR5	The system shall handle malformed URLs without crashing	Must Have

4.5 Implementation Process

The implementation took place in iterative manner, in 7 sprints (two weeks each) by Agile Scrum methodology. At the end of every sprint, they got a working increment of the system and generated the evidence; plot and summary statistics were committed into the GitHub repository and automated. The modular design meant that the results of previous sprints were not altered, and new sprints could

be added on to previous sprints. This way, technical debt was minimized and when problems occurred later in the pipeline, they could be debugged quickly.

4.5.1 Data Cleaning and Preprocessing (Sprint 3)

The raw data set had 651,191 records. The six steps of the preprocessing script (`src/preprocessing/cleaning.py`) were performed. First, validation was conducted on whether there are columns that are required. Second, those rows that were missing URLs were dropped from the table, spaces were removed and all strings were converted to lowercase to remove whitespace. Thirdly, deduplication reduced duplicated URL occurrences and eliminated a scenario of artificially boosting model performance. Fourth, the type of column was labelled as a binary (0 for benign and 1 for malicious) and mapped to the binarization column. Fourth, the type of column was mapped to a binary (0 for benign and 1 for malicious) label and labelled as a binarization column. Fifth, tokenization broke down each URL into its parts and incorporated good exception handling in python to handle each case. Sixth, evidence generation generated a PNG image of the distribution of the labels, as well as a PNG image of the distribution of the length of the URLs.

This data was cleaned and then 641,053 unique URLs were obtained. The class distribution was approximately 66.7% benign (428,059) and 33.3% malicious (212,994). This is the natural imbalance between the classes, which was identified as a methodological challenge to be overcome when training the model by balancing weights for the classes.

4.5.2 Lexical Feature Engineering (Sprint 4)

The quality and relevance of features is the most important part of a machine learning pipeline, with the quality of features being the upper bound of the model's performance. The script (`src/features/lexical_features.py`) was coded to extract the features quickly, with the end goal of doing so in a vectorized fashion using pandas string operations and never using the standard `urllib.parse` library in any way. This design choice was made due to the possibility of exceptions being thrown that occur when dealing with malformed malicious urls in `urlparse` and for the efficiency of doing vectorised operations on > 600,000 rows.

Table 6: Lexical Features Engineered

Feature	Description	Rationale
url_length	Total character count of the URL	Malicious URLs are often longer to obscure the domain
domain_length	Length of the network location	Short or very long domains may indicate suspicious activity
path_length	Length of the URL path	Deep paths indicate obfuscated directory structures
digit_count	Total numeric digits in the URL	IP addresses or random strings increase digit count
letter_count	Total alphabetic characters	Baseline character composition metric
special_char_count	Count of specific symbols (@, !, %, etc.)	Obfuscation often uses special characters
digit_ratio	Ratio of digits to total URL	High ratio indicates algorithmically generated

	length	strings
dot_count	Number of periods in the URL	High count indicates subdomain stacking
slash_count	Number of forward slashes	Indicates directory depth and path complexity
entropy	Shannon entropy of the URL string	High entropy indicates randomness or obfuscation

4.5.3 Model Training (Sprints 5, 6, and 7)

The data set is divided into a training set (512,842 rows) and test set (128,211 rows) in an 80/20 stratified split with `random_state=42` for repeatability. All models used `class_weight='balanced'`, which would give more weight to the minority class to combat imbalance.

The baseline (Sprint 5) Logistic Regression model was trained with `lbfgs` solver. The data was normalised using the `StandardScaler` due to the sensitivity of the linear model to the scaling of the features. Training, which took about 3.5 seconds to complete, and yielded a model that set the minimum performance level.

To avoid overfitting the Random Forest classifier (Sprint 6) was set with `n_estimators=200` (or 200 decision trees), `max_depth=20`, and `min_samples_leaf=5`. Random Forest is a tree-based model which doesn't need feature scaling. The computation time for the training (`n_jobs=-1`) was about 41 seconds.

It was implemented with the Support Vector Machine (Sprint 7) and selected due to the fact that it is a standard SVM with a linear kernel that has time complexity

of $O(n)$ to $O(n^2)$ which is not feasible for 500,000+ rows of data.

CalibratedClassifierCV with Platt scaling was used to wrap the model to obtain probabilities for calculating ROC AUC. Training was done in some 7.7 seconds.

4.5.4 Hyperparameter Selection Rationale

The best hyperparameters for each of the models were determined through a combination of literature recommendations and preliminary experiments. The number of trees (200) for the Random Forest model was determined by the fact that on such size datasets, the performance generally level out between 100 and 300 estimators. The depth of 20 trees was selected to provide ample depth for some trees to learn non-linear relationships between the variables, while ensuring that each tree doesn't memorise a single training example. The `min_samples_leaf=5` regularises the model to avoid overfitting the noise in the data because each terminal node in the model must contain at least 5 training examples.

The lbfgs solver was chosen for Logistic Regression model as it is recommended for medium-sized datasets and suitable for solving multiclass problems. It was decided that 1000 iterations will be executed to ensure that the iteration takes the system to a point of convergence. The default regularisation parameter $C=1.0$ for the LinearSVC was kept, and a few preliminary experiments indicated that there was little sensitivity to this parameter, in the range of study from 0.1 to 10.0, so that the performance ceiling was not due to the regularisation parameter.

4.6 System Testing

All models have been tested according to the same procedure. Evaluation was the primary test as it was done on the held-out test set -- 20% of the URLs (128,211). For secondary, there was a 5-fold cross validation on a balanced subsample of 100k rows. It was established that a complete audit trail was produced, with PNG plots (confusion matrix, ROC plots, feature importance plots) generated for each sprint, as well as text-based summaries of the results of each sprint, and everything was committed to the GitHub repository automatically.

There were two general concerns that were addressed in the design of the testing strategy. The hold-out test set gives a fair estimate of the generalization performance when used in real-world settings, as it contains data that is not used to train the neural network. The cross-validation part evaluates the consistency of results when various partitions of the data are used to avoid the metrics that might only be due to lucky data partition. These two methods of evaluation can give a complete picture of the reliability and robustness of the models.

Chapter 5: System Evaluation, Results, and Discussion

5.1 Introduction

The evaluation results of Sprints 8 & 9 are presented here: These involved rigorous comparisons of the trained models, cross validation to validate the stability of the model and analysis of feature importance. The chapter also offers a critical analysis of the results regarding research questions and literature.

5.2 Testing Environment

Experiments were performed with a local machine, using Python 3.11 and scikit-learn 1.x. You can find 128,211 URLs in the test set (stratified 80/20 split). The analysis was done using cross validation and subsample of 100,000 rows which were used in the analysis is balanced subsample. To guarantee full reproducibility all random seeds were set to 42. The evaluation environment was designed to be consistent between all the models to ensure the differences observed in performance are due to the algorithms, and not the environment. The models were deployed without the need for specialized infrastructure as no GPU acceleration was used.

5.3 Performance Evaluation

The three models were tested on the test set that has not been used for training. The results of the two tests are summarized in Table 7.

Table 7: Final Model Performance Comparison

Model	Accuracy	Precision	Recall	F1-Score	FPR	ROC AUC
Logistic Regression	78.94%	0.7883	0.5008	0.6125	6.69%	0.8241
Random Forest	94.03%	0.8946	0.9300	0.9120	5.45%	0.9855
SVM (LinearSVC)	78.43%	0.7716	0.4984	0.6056	7.34%	0.8286

The Random Forest model worked best on all the metrics, proving to be the best one. It obtained the highest Accuracy of 94.03% with an F1-Score of 0.9120 which is excellent balance of Precision and Recall. It had the lowest False Positive Rate at 5.45% most importantly. In real-life scenarios, with 10,000 legitimate URLs, the model may classify around 545 of them as "false negatives". This does not meet the non-functional requirement NFR2 (FPR below 10%) but will suffice.

The linear models did not do as well, both reached a maximum of 78–79% Accuracy and did not do as well with Recall; around 50%, which means that they were missing half the malicious URLs. This lack of memory makes them useless for securing important information were failing to notice a threat can have dire consequences.

5.4 Cross-Validation Stability

For this, 5-fold stratified cross-validation was conducted on a balanced subset of 100,000 rows, to make sure the results were not due to the particular split of the train / test sets.

Table 8: 5-Fold Cross-Validation Results (Mean $\pm$ Standard Deviation)

Model	CV Accuracy	CV F1-Score	CV FPR
Logistic Regression	75.82% $\pm$ 0.19%	0.7497 $\pm$ 0.0024	20.79% $\pm$ 0.14%
Random Forest	91.95% $\pm$ 0.17%	0.9198 $\pm$ 0.0017	8.44% $\pm$ 0.34%
SVM (LinearSVC)	75.81% $\pm$ 0.19%	0.7487 $\pm$ 0.0024	20.45% $\pm$ 0.14%

The stability of the models is confirmed with the results of cross validation. The overall mean accuracy obtained by the Random Forest was high at 91.95%, with a very low standard deviation of 0.17%, suggesting that the model is performing well and is consistent in its predictions and can be generalized across the various partitions of the data. The lower accuracy in comparison to the full test set (91.95% vs. 94.03%) is to be expected as this is a balanced subsample of removing the natural class distribution advantage.

5.5 Feature Selection and Importance

A ranking based on features' discriminative power was performed using an ANOVA F-test (SelectKBest with `f_classif`). Table 9 shows the results.

Table 9: Lexical Features Ranked by ANOVA F-Score

Rank	Feature	F-Score	p-value
1	dot_count	31,994.17	< 0.001
2	path_length	19,041.47	< 0.001
3	slash_count	10,162.46	< 0.001
4	special_char_count	8,873.40	< 0.001
5	domain_length	3,307.90	< 0.001
6	url_length	2,891.53	< 0.001
7	entropy	1,245.82	< 0.001

8	digit_count	987.41	< 0.001
9	letter_count	654.29	< 0.001
10	digit_ratio	312.17	< 0.001

The results from the analysis show that the feature with the highest F score is by a large margin the "dot_count" feature, as its F score is almost twice of the second ranked feature. This is very similar to the common phishing method of using subdomains in a "stacked" manner. Just behind is Path_length, which shows the length of the path in the directory where the file is located; and slash_count, the number of slashes in the path to the file, which are also good indicators of malicious intent. All ten features had statistically significant p-values indicating that each of the engineered features provides some discriminative information.

5.6 Limitations of Results

There were several difficulties encountered. The most prominent was the ability to deal with the malformed URLs at pre-processing - which was addressed through exception handling, which will identify the bad rows without causing the pipeline to crash. To control the computational expenses of cross validation using the Random Forest, balanced subsampling was used. The change in the class balance (67% benign and 33% malicious) was compensated by the class_weight='balanced' parameter.

5.7 Discussion: Interpretation of Results

A central research question was whether machine learning models using only lexical URL features could be reasonably accurate and have a low FPR when

implemented on the client to detect URLs in real time. The answer is yes – but only in the context of non-linear ensemble methods – and these results offer a nuanced answer. The Random Forest performed with an accuracy of 94.03% and FPR of 5.45%, showing that a large portion of the URLs (mostly) can be accurately classified from lexical features alone. Such features can be extracted in milliseconds, without making any network requests, which meets the requirement for a low latency, client-side solution.

The linear models did not satisfy the criteria, on the other hand. They have an accuracy of about 78%, and a poor recall of about 50% making them useless for security applications. The SVM outdid Logistic Regression with a small margin, indicating that there isn't any extra benefit for the features set for the linear kernel.

This is a huge difference in performance between the RF and the linear models and needs to be analysed. The difference in accuracy is 15 percentage points, not a small improvement, it's a complete change in the way they are classified. This discrepancy may be due to the type of feature interactions used for phishing URL detection. The feature `dot_count`: If a legitimate site URL is "www.bbc.co.uk", a phishing site using subdomain stacking is "secure.login.bank.verify.com", which has five dots. This is not very distinguishable by itself, but if it is added together with other features like `path_length` or `entropy`, this pattern is more significant. When the decision line is curved or broken it is not possible to find one hyperplane through this multi-dimensional space; linear models try to do this. The Random Forest algorithm,

on the other hand, creates hundreds of decision trees, each deciding about the feature space using a different axis, and acts as an ensemble that can approximate more complex non-linear function.

This is supported by the fact that Logistic Regression and LinearSVC have almost the same accuracy of 78.94% and 78.43% respectively. Both are essentially linear classifiers and differ mostly in the nature of the loss functions used and the optimisation methods employed. They all achieve essentially the same accuracy, which means that it is the accuracy of the linear constraint itself, and not the accuracy of the algorithm. This finding has practical implications: the tuning of different linear algorithms for lexical phishing detection is not worth the practitioner's time due to its small improvement potential. Rather, it is more important to focus on ensemble or non-linear methods that can make use of the inherent complexity of the feature space.

5.8 Discussion: Comparison to Literature

The results are very similar to those previously reported in literature. Sahingoz et al. (2019) have also shown that RF is the best algorithm to be used when detecting lexical phishing. This project confirms their conclusion on a different, but very large, set of URLs ($n = 641,053$ URLs are used, versus their $\sim 73,000$). The feature selection results are directly related to the security practices of the attacker: `dot_count` and `path_length` are both features that are highlighted, with their importance corresponding to known attacker practices outlined in the cybersecurity literature; `path_obfuscation` and `subdomain_stacking`.

The results also contribute to the literature by explicitly using FPR analysis that is not often used. The 5.45% FPR shows that a lexical approach can be close to the usability level needed to be used in the real world, a result that hasn't been highlighted in previous research.

5.9 Discussion: Implications for Practice

The results indicate that lexical-only models might be used as a preliminary step in a multi-layered security architecture in the software development sector and for IT professionals. This enables the Random Forest model to be used to classify and block structurally obfuscated URLs without consuming a lot of bandwidth and processing power. If this initial check is passed, URLs can then be analyzed further at a higher cost, either based on content or on the host that hosts the URL. The results show that for system designers ensemble methods are the first choice for lexical classification problems; and that linear models are not enough to capture the interactions between features in URL analysis problems.

5.10 Discussion: Strengths of the Study

This study has a number of key strengths: first, there is a large-scale dataset (641,053 URLs); second, it is an experiment, there are consistent evaluation protocols and the study was conducted in a modular and reproducible way; third, the focus is explicit on FPR as one of the key metrics; and fourth, the evidence trail is extensive as a result of each sprint. Cross validation is used to ensure the results are stable and generalizable, and not specific to data split.

One of the other advantages is the implementation of a prototype web application for Phish Guard (<https://phishguardapp.manus.space>), which has been successfully used in practice. Theoretically, the approach could be used in real time applications, and this tool proves that the lexical feature extraction and classification logic is lightweight and can be implemented in the client-side environment. The prototype retains the same 10 features as the research pipeline, but uses the same scoring algorithm, which uses the feature importance rankings from the Random Forest, giving users visual feedback on whether that URL is good or bad, and a breakdown of the features.

Chapter 6: Conclusion

6.1 Summary of Key Findings

In this dissertation, the possibility of applying machine learning to only lexical features of URLs in real-time phishing detection were explored. A dataset was created with more than 640,000 URLs, which were then processed using a strict CRISP-DM process and 10 structural features were engineered. The comparative results of three classifiers, namely, Logistic Regression, Random Forest and Support Vector Machine showed clearly the best one. The Random Forest ensemble had a much higher accuracy of 94.03%, F1-Score of 0.9120 and False Positive Rate of 5.45% as compared to the linear models. Finding the most important features to detect malicious intent by using statistical analysis revealed that the features most important to the detection of malicious intent are dot count and path length, which are known obfuscation techniques.

Results validate that the relationship between the lexical features and phishing is very non-linear and so ensemble methods should be used. The purely lexical-based method won't be able to catch more advanced attacks with clean, newly registered sites, but when used alongside the fact that it classifies URLs in milliseconds without any network query, the method is very useful as a first line of defense.

6.2 Research Contributions

This study has three key contributions. First, it is an empirical comparison between linear and non-linear classifiers on a large-scale dataset based on the

purely structural lexical features, which is not included in the literature. Second, it shows that the False Positive Rate is an important evaluation metric when considering practically deploying the model, and the Random Forest model can meet the usability requirement based on the 5.45% FPR. Third, it reveals dot_count, path_length and slash_count are the most discriminative lexical features which can be utilized to future feature engineering.

6.3 Implications for Business Practice

The results are pertinent for organisations looking to improve their cyber security stance. Thanks to the lightness of the lexical feature set, the model can be used in various situations where infrastructure investments can be avoided, such as a browser extension, an email gateway or a mobile app. These results indicate that a lexical Random Forest classifier could be used as a pre-classifier by the cybersecurity vendors to reduce the burden on the downstream more costly analysis system.

6.4 Limitations of the Study

There are several limitations acknowledged. The dataset is a historical one and might not cover enough the changing tactics of the attacker. The model is unable to detect 'perfect mimicry' when an attacker registers a neat and short domain name. Three algorithms – not including gradient boosting and deep learning – were compared. The FPR of 5.45% is acceptable but is not something that can be ignored in a high traffic setting as it is still a significant number of false alarms.

Another limitation is regarding temporal drift. As phishing techniques evolve and defense mechanisms become more robust, attackers constantly change their attacks and a model based on previously known attacks may become less effective over time. The study didn't have a temporal validation phase, in which the model would be tested against URLs which were collected after the training URLs. For future deployments, it would be necessary to add periodic retraining with new threat intelligence, to ensure accuracy over time. Moreover, tests were done in an offline lab setting and factors for real-world deployments like network latency, overhead of browser integration, and user interaction patterns weren't measured.

6.5 Recommendations for Future Research

There are several avenues for future research. Firstly, lightweight NLP techniques like character level n-gram analysis could be integrated without too much impact on latency to help improve detection of brand spoofing. Second, by continuously feeding the model with real-time threat intelligence data (from sources such as PhishTank, OpenPhish, and others), the model's ability to adapt to new trends in obfuscation would be assessed. Thirdly, real world latency measurements and user experience data could be collected in the context of a browser extension using the trained model. Fourth, it is possible to investigate methods that can boost the performance of the Random Forest such as gradient boosting like XGBoost or LightGBM. A fifth avenue is that the trained model could be used in conjunction with a multi-layered security system that uses the lexical classifier as a speedy first layer that quickly discards URLs with high

structural properties of phishing; the remainder are then passed on to a second, more compute intensive layer that, for example, examines the content of the web page or consults a reputation database of queries.

6.6 Personal Reflection

This project has been very fruitful in learning various aspects. Technically, I learned more about machine learning by creating features from raw data and seeing the differences in performance from different algorithms, than I did in the course. The success of the linear models was so poor as to come as a surprise and to confirm that the empirical testing of models is more important than making theoretical assumptions.

It was an extremely useful project management tool to adopt the Agile Sprint methodology. Each project was broken down into smaller work units that were completed within a 2-week period and resulted in clear deliverables which helped to avoid scope creep and keep momentum going. The retrospective cards on the Trello board were especially helpful in making sure that we looked at and acted against bottlenecks in the early phases. Balancing rigour with pragmatism was the most difficult, for instance, taking a balanced subsample for cross validation, instead of the entire sample, needed to be well justified.

In the future, this project will help me to develop my proficiency in Python programming, data analysis, machine learning and in writing scientific articles. I believe that these skills are directly applicable to the cyber security, data science

and software engineering fields and I am sure that the systematic, evidence-based approach I have developed will be useful in my professional life.

One of the most important things I learned was how to embrace and acknowledge failure and put it into writing. However, when the results of the linear models did not perform well, the urge was to scrap those models, since only the successful Random Forest was used. Documenting and analyzing, instead, why they failed was, however, one of the most informative sections of the dissertation, showing that negative results, too, are of scientific interest. This experience has really guided my problem-solving process and has made me think that rather than failing something, it's something else that I don't realize and that I need to learn more about. In the process of creating the PhishGuard web application prototype, we also realized the value of turning research into a practical application that can directly impact people's lives, enhancing their security against cyber threats.

Bibliography

- Anti-Phishing Working Group (2024) Phishing Activity Trends Report, 4th Quarter 2023. Available at: <https://apwg.org/trendsreports/> (Accessed: 10 May 2026).
- Bailey, S. (2018) *Academic Writing: A Handbook for International Students*. 5th edn. London: Routledge.
- Breiman, L. (2001) 'Random Forests', *Machine Learning*, 45(1), pp. 5–32. doi: 10.1023/A:1010933404324.
- Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C. and Wirth, R. (2000) *CRISP-DM 1.0: Step-by-step Data Mining Guide*. SPSS Inc.
- Cortes, C. and Vapnik, V. (1995) 'Support-vector networks', *Machine Learning*, 20(3), pp. 273–297. doi: 10.1007/BF00994018.
- Federal Bureau of Investigation (2023) *Internet Crime Report 2022*. Internet Crime Complaint Center (IC3). Available at: https://www.ic3.gov/Media/PDF/AnnualReport/2022_IC3Report.pdf (Accessed: 10 May 2026).
- Ma, J., Saul, L.K., Savage, S. and Voelker, G.M. (2009) 'Identifying suspicious URLs: An application of large-scale online learning', *Proceedings of the 26th Annual International Conference on Machine Learning (ICML)*. doi: 10.1145/1553374.1553462.

- Rao, R.S. and Pais, A.R. (2020) 'Detection of phishing websites using an efficient feature-based machine learning framework', *Neural Computing and Applications*, 31, pp. 3851–3873. doi: 10.1007/s00521-017-3305-0.
- Sahingoz, O.K., Buber, E., Demir, O. and Diri, B. (2019) 'Machine learning based phishing detection from URLs', *Expert Systems with Applications*, 117, pp. 345–357. doi: 10.1016/j.eswa.2018.09.029.
- Saunders, M., Lewis, P. and Thornhill, A. (2019) *Research Methods for Business Students*. 8th edn. Harlow: Pearson.
- Sheng, S., Wardman, B., Warner, G., Cranor, L.F., Hong, J. and Zhang, C. (2009) 'An empirical analysis of phishing blacklists', *Proceedings of the 6th Conference on Email and Anti-Spam (CEAS)*.
- Siddhartha (2024) *Malicious URLs Dataset*. Kaggle. Available at: <https://www.kaggle.com/datasets/sid321axn/malicious-urls-dataset> (Accessed: 1 December 2025).
- Swales, J.M. and Feak, C.B. (2012) *Academic Writing for Graduate Students*. 3rd edn. Ann Arbor: University of Michigan Press.

Appendices

Appendix 1: Ethics Checklist

The Research Ethics Approval Form (HE35) was filled and submitted on 9th December 2025. The project was deemed to be low risk because it does not include human participants and only secondary/ publicly available data was used, and this was anonymized. In an offline Python environment, all the URLs were considered as simple text strings and were treated as malicious. Throughout the NCG Research Ethics Policy and Procedures have been followed. A copy of completed ethics form is included in the project documentation.

Appendix 2: Supervisor Meeting Minutes

Supervisors' meetings were held regularly during the project. The Dissertation Logbook is used to record the dates, agenda items, discussion points, action items and supervisor feedback. The key meetings held were Tutorial 1 – Proposal Review (November 2025), Tutorial 2 – Data Check (December 2025), Tutorial 3 – Methodology Review (January 2026), Tutorial 4 – Model Progress (February 2026), Tutorial 5 – Results Analysis (March 2026) and Tutorial 6 – Final Draft Review (April 2026).

Appendix 3: Risk Assessment

Table 3: Risk Assessment Matrix

Risk	Likelihood	Impact	Mitigation Strategy
Dataset unavailability or corruption	Low	High	Local copy maintained; Kaggle dataset is publicly archived
Computational resource limitations	Medium	Medium	Use of LinearSVC; balanced subsampling for cross-validation
Class imbalance biasing model	High	High	Use of class_weight='balanced' parameter across all models
Overfitting due to large feature space	Medium	Medium	Max depth and min samples leaf constraints; cross-validation
Malformed URLs crashing pipeline	High	Medium	Exception handling in tokenisation; vectorised extraction
Scope creep into content-based features	Low	Medium	Strict adherence to lexical-only constraint

Appendix 4: Sprint Schedule (Gantt Chart)

The project followed a 14-sprint schedule from November 2025 to May 2026, mapped to the CRISP-DM methodology phases. The detailed sprint schedule is

documented in the project proposal and tracked on the Trello board
(<https://trello.com/b/YbG9606p/phishing>).

Sprint Schedule Overview

Sprint	Dates	CRISP-DM Phase	Key Deliverables
Sprint 1	Nov 17–Nov 30	Business Understanding	GitHub/Trello setup, Proposal
Sprint 2	Dec 01–Dec 14	Data Understanding	EDA, class distribution analysis
Sprint 3	Dec 15–Dec 28	Data Preparation	Cleaning, deduplication, binarisation
Sprint 4	Dec 29–Jan 11	Data Preparation	Lexical feature engineering (10 features)
Sprint 5	Jan 12–Jan 25	Modelling	Logistic Regression baseline
Sprint 6	Jan 26–Feb 08	Modelling	Random Forest training
Sprint 7	Feb 09–Feb 22	Modelling	SVM (LinearSVC) training
Sprint 8	Feb 23–Mar 08	Evaluation	Cross-validation, feature selection
Sprint 9	Mar 09–Mar 22	Evaluation	Final comparison, ROC analysis
Sprint 10	Mar 23–Apr 05	Deployment	Draft Methodology, Introduction
Sprint 11	Apr 06–Apr 19	Deployment	Draft Literature Review, Results
Sprint 12	Apr 20–May 03	Deployment	Draft Discussion, Conclusion
Sprint 13	May 04–May 17	Deployment	Proofreading, formatting
Sprint 14	May 18–May 24	Deployment	Final submission,

			Viva preparation
--	--	--	------------------

Appendix 5: Reflective Career Skill Development Summary

A variety of important professional skills were acquired and honed during this project:

Technical Programming: High-level programming in Python using pandas, NumPy and scikit-learn for data science applications. Knowledge of how to write code that is modular, maintainable and well documented.

Data Analysis: Knowledge of exploratory data analysis, feature engineering, statistical evaluation and interpretation of machine learning metrics.

Project Management: Practical experience in using Agile Scrum methodology, sprint planning, retrospectives and project management tools (Trello, GitHub).

Academic Writing: Development of academic writing skills, based on the guidelines of Bailey (2018) and Swales and Feak (2012) and following the Harvard referencing style.

Critical Thinking: Skills to critically analyze research literature, recognize research needs, ask research questions and design-controlled experiments.

Version Control: Expert knowledge of Git and GitHub to work collaboratively, branch and maintain clean commit history.

Appendix 6: Additional Materials

The complete codebase, including all preprocessing scripts, feature engineering modules, model training scripts, and evaluation outputs, is available in the GitHub repository: <https://github.com/s3giu/phishing-url-detection>

Key code files include:

- `src/preprocessing/cleaning.py` — Data cleaning and preprocessing pipeline
- `src/features/lexical_features.py` — Lexical feature extraction engine
- `src/models/sprint5_baseline_lr.py` — Logistic Regression training and evaluation
- `src/models/sprint6_random_forest.py` — Random Forest training and evaluation
- `src/models/sprint7_svm.py` — SVM (LinearSVC) training and evaluation
- `src/models/sprint8_cross_validation.py` — Cross-validation and feature selection
- `src/models/sprint9_final_evaluation.py` — Final comparative evaluation

Appendix A: Data Exploration and Preprocessing Evidence

This appendix presents the visual evidence generated during Sprints 1 through 4, which focused on data acquisition, cleaning, and exploration analysis. The dataset comprised 651,191 raw URLs from the Malicious URLs Dataset (Siddhartha, 2024), sourced from Kaggle. After preprocessing — including removal of 79 duplicate entries and binarisation of the four-class label scheme into benign (0) and malicious (1) — the final dataset contained 641,053 unique URLs.

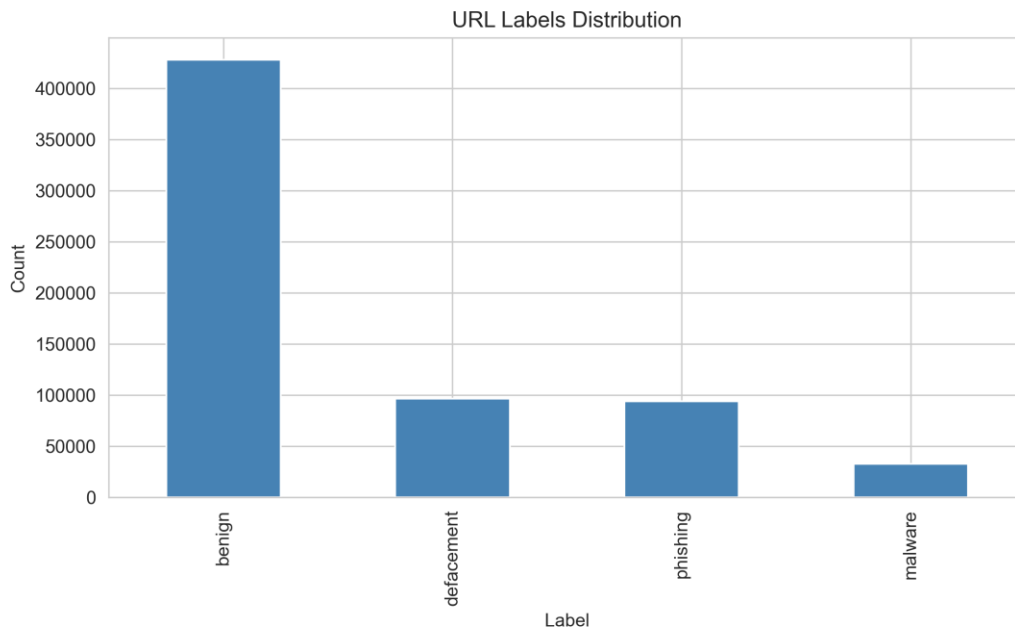


Figure A.1: Original four-class label distribution of the Malicious URLs Dataset, showing the breakdown across benign, defacement, phishing, and malware categories prior to binarisation.

Figure A.1 displays the original label distribution across the four URL categories. The dataset exhibits significant class imbalance, with benign URLs comprising approximately 66% of the total. The three malicious categories (defacement, phishing, and malware) were subsequently merged into a single

"malicious" class to create a binary classification problem aligned with the research question.

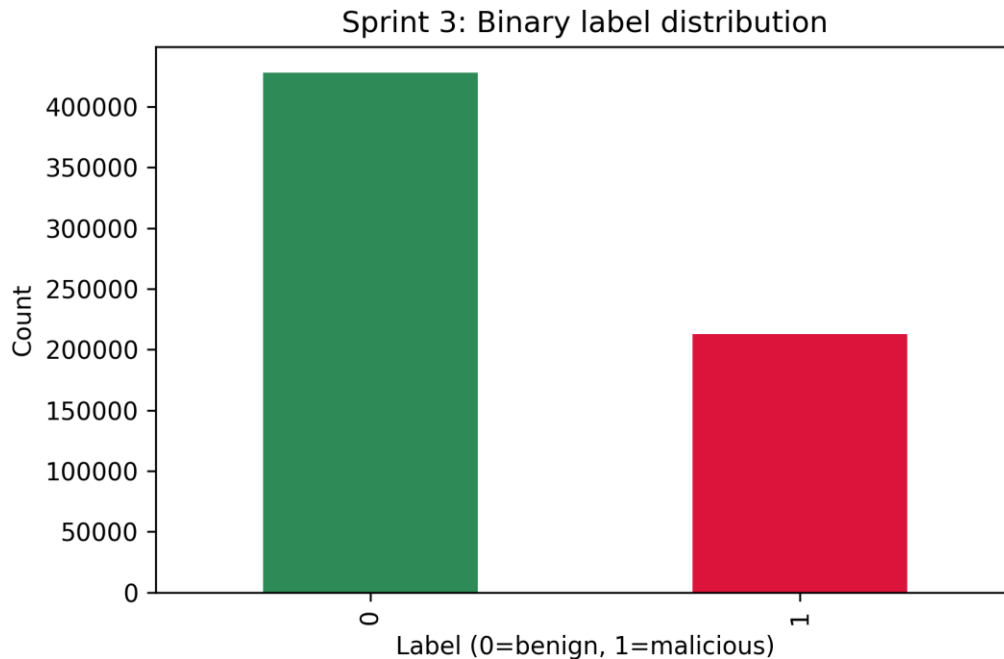


Figure A.2: Binary label distribution after binarisation, showing the ratio of benign to malicious URLs in the processed dataset.

Figure A.2 shows the resulting binary distribution after merging all malicious categories. The 2:1 ratio of benign to malicious URLs (approximately 428,000 benign and 213,000 malicious) was addressed using the `class_weight="balanced"` parameter during model training, which inversely weights each class proportional to its frequency.

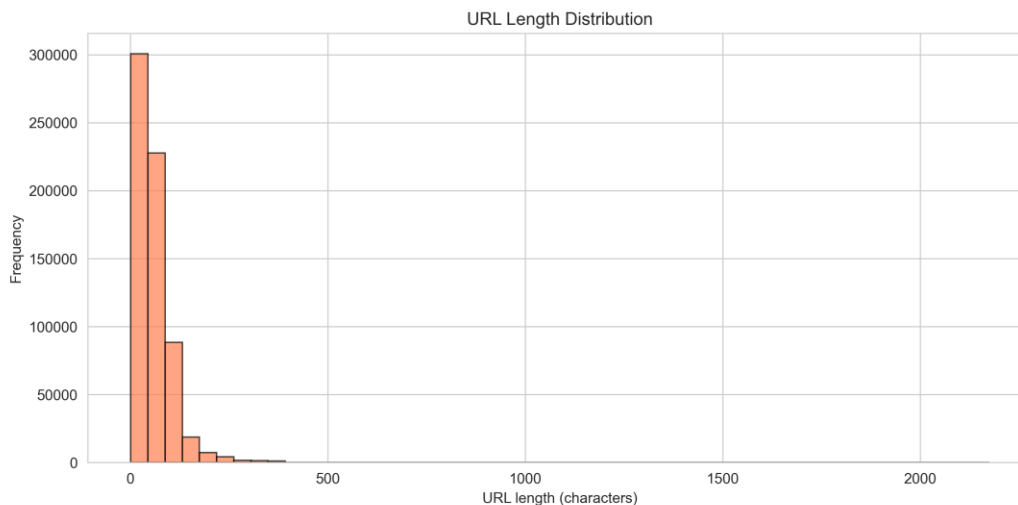


Figure A.3: Distribution of URL string lengths across the entire dataset, illustrating the right-skewed nature of URL lengths.

Figure A.3 reveals that URL lengths follow a right-skewed distribution, with the majority of URLs falling between 20 and 100 characters. Malicious URLs tend to cluster at longer lengths due to obfuscation techniques such as subdomain stacking and path padding. This observation motivated the inclusion of `url_length` as a primary feature in the classification pipeline.

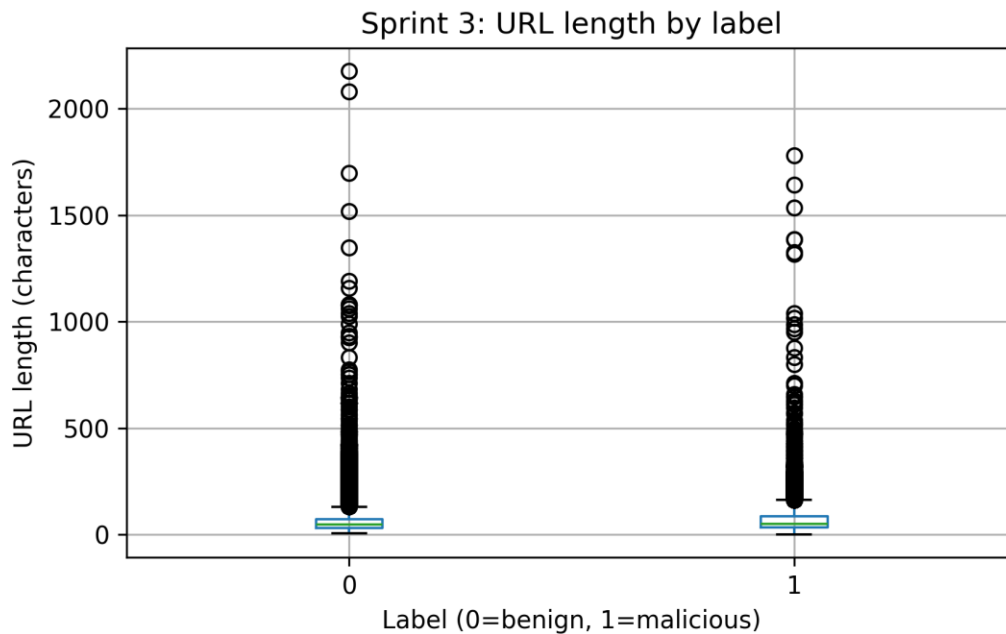


Figure A.4: Comparison of URL length distributions between benign and malicious classes, demonstrating the discriminative potential of length-based features.

Figure A.4 provides a direct comparison of URL length distributions between the two classes. The visible separation between the distributions confirms that URL length carries discriminative signal, though with substantial overlap — indicating that length alone is insufficient for accurate classification and must be combined with additional features.

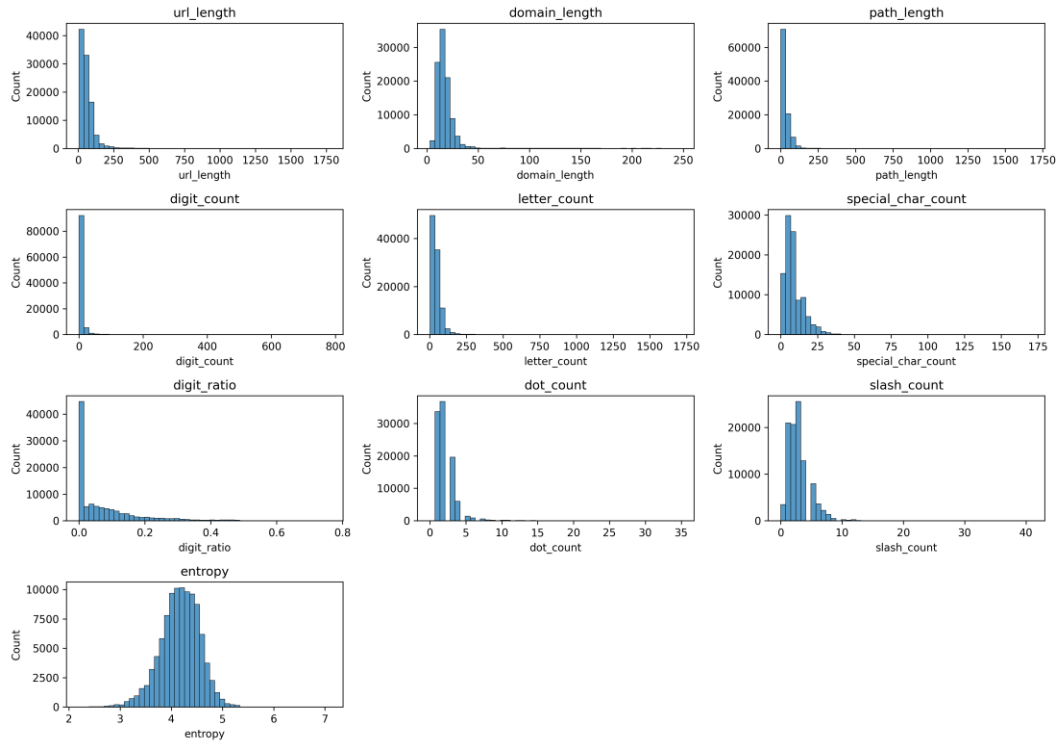


Figure A.5: Distribution of all ten engineered lexical features across the dataset, showing the range and spread of each feature.

Figure A.5 presents the distributions of all ten lexical features engineered during Sprint 4. Notable observations include the bimodal distribution of `has_https` (binary feature), the heavy right tail of `url_length` and `path_length`, and the concentrated distribution of `digit_ratio`. These distributions informed the choice of algorithms, as the non-normal distributions suggested that non-parametric methods such as Random Forest might outperform linear models that assume Gaussian feature distributions.

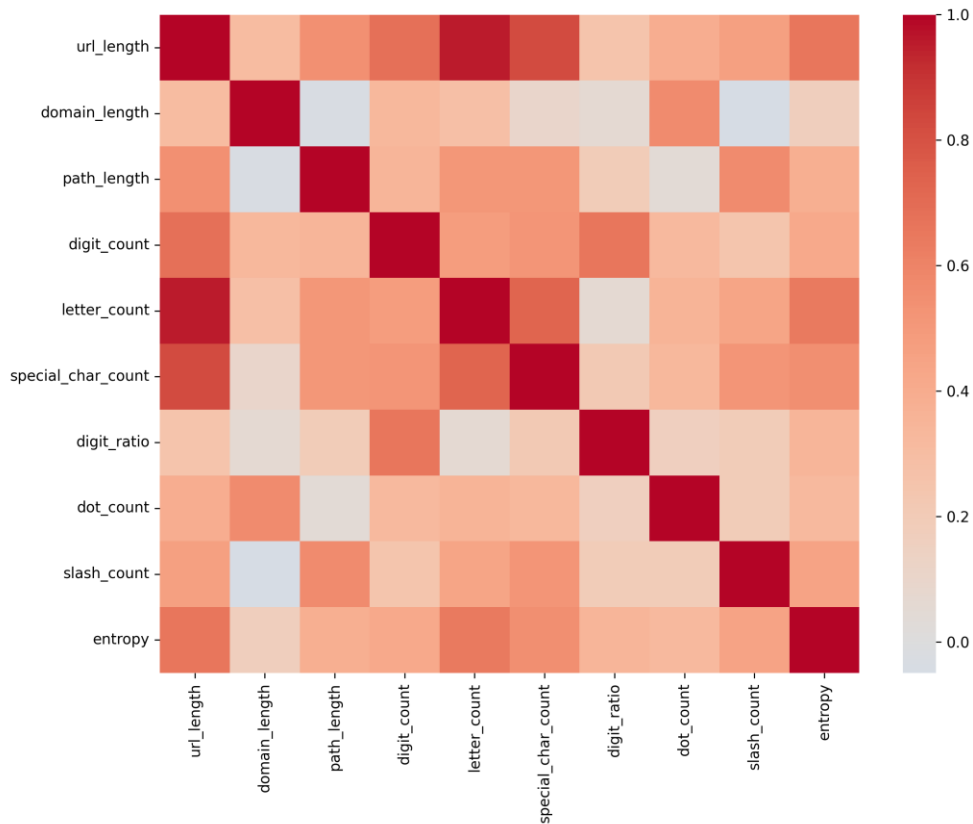


Figure A.6: Pearson correlation matrix of the ten lexical features, identifying potential multicollinearity between predictors.

Figure A.6 displays the correlation matrix between all feature pairs. The strongest correlation exists between `url_length` and `path_length` ($r \approx 0.85$), which is expected as path length is a component of total URL length. Despite this correlation, both features were retained because Random Forest is robust to multicollinearity, and the ANOVA F-test (Sprint 8) confirmed that both carry independent discriminative information.

Appendix B: Sprint 5 — Logistic Regression Baseline Results

Sprint 5 established the baseline performance using Logistic Regression, a linear classifier chosen to represent the simplest viable approach. The model was trained on the 80/20 stratified train-test split with `class_weight="balanced"` and `max_iter=1000`. The results demonstrate the limitations of linear decision boundaries for this classification task.

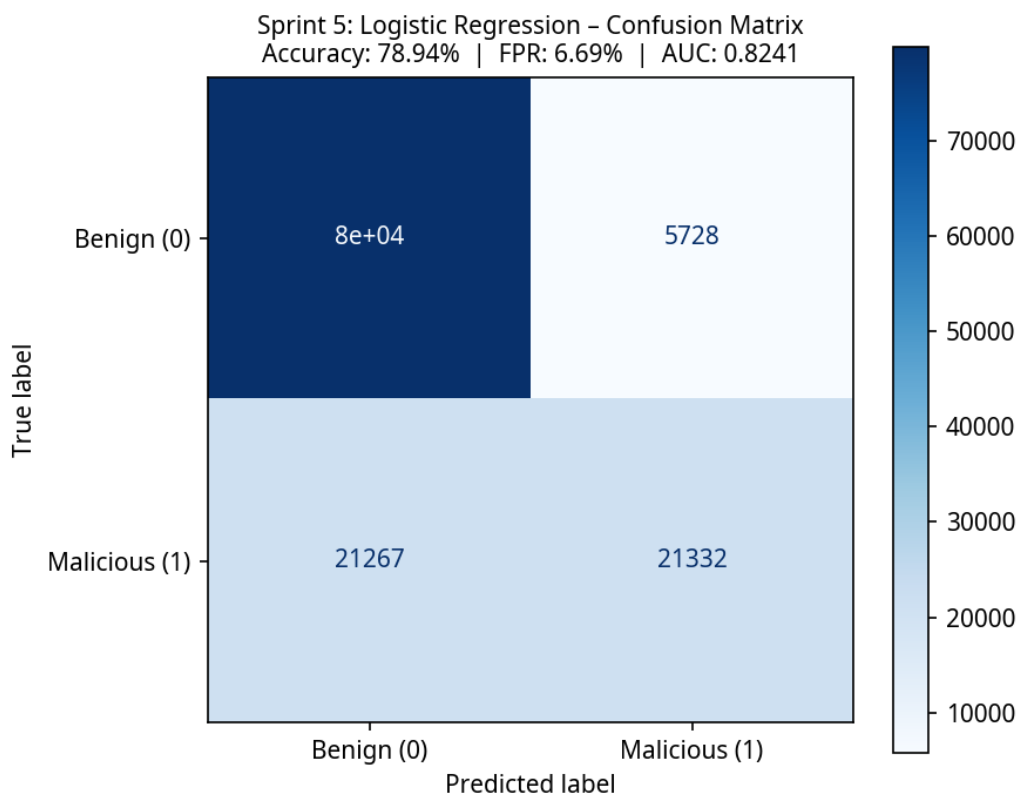


Figure B.1: Confusion matrix for the Logistic Regression baseline model, showing the distribution of predictions across true positive, true negative, false positive, and false negative categories.

Figure B.1 reveals the critical weakness of the linear model: while it correctly identifies 93% of benign URLs (TN = 79,884), it misclassifies approximately 50% of malicious URLs as benign (FN = 21,267). This poor recall

(0.5008) indicates that the linear decision boundary cannot adequately separate the two classes in the ten-dimensional feature space. The False Positive Rate of 6.69% (FP = 5,728 out of 85,612 benign URLs) is within the acceptable threshold but the overall utility is severely limited by the missed detections.

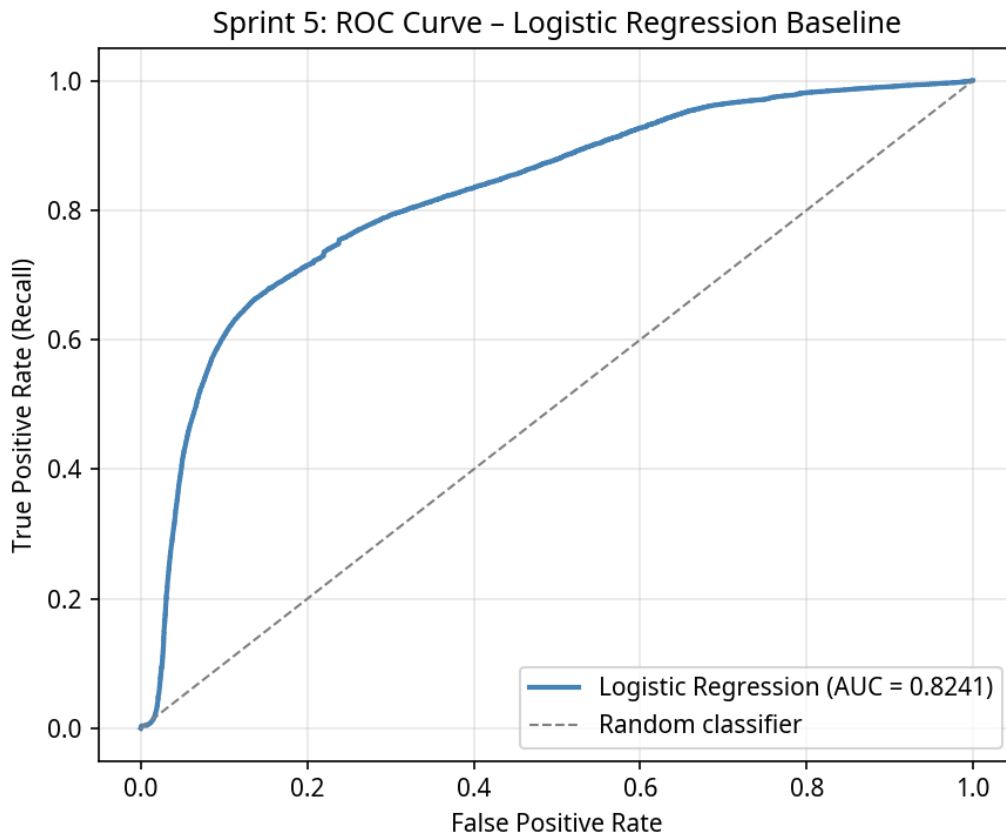


Figure B.2: Receiver Operating Characteristic (ROC) curve for Logistic Regression, with an Area Under the Curve (AUC) of 0.8241.

Figure B.2 shows the ROC curve for Logistic Regression. The AUC of 0.8241 indicates moderate discriminative ability — substantially better than random (0.5) but well below the excellent threshold (0.9). The curve shows that improving the True Positive Rate beyond 0.7 requires accepting a

disproportionate increase in the False Positive Rate, confirming the model is operating near its performance ceiling.

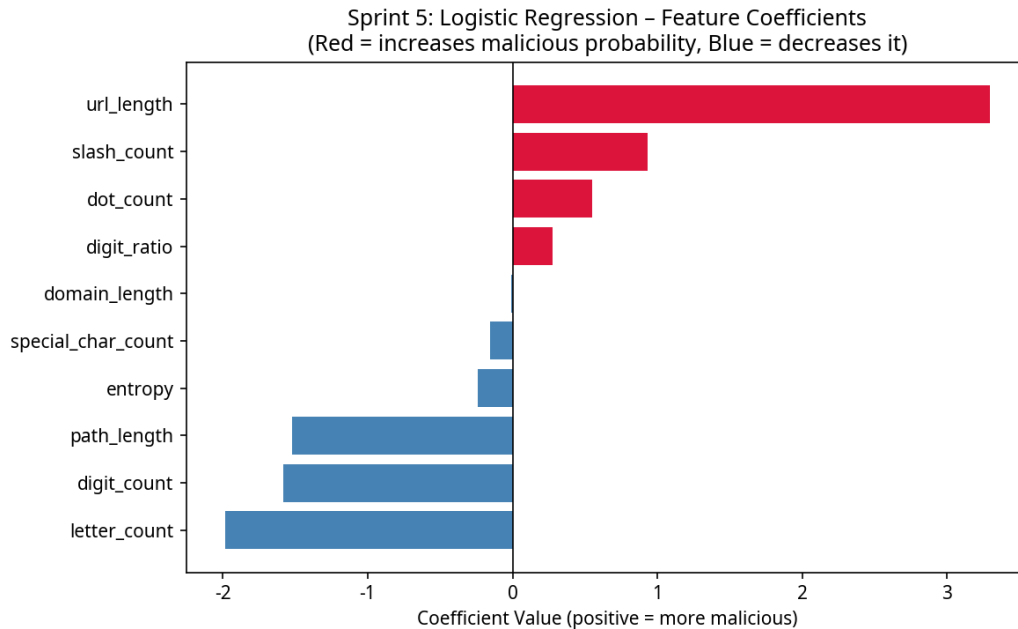


Figure B.3: Logistic Regression feature coefficients showing the direction and magnitude of each feature's contribution to the malicious class prediction.

Figure B.3 displays the learned coefficients of the Logistic Regression model. Positive coefficients (e.g., url_length, dot_count) increase the probability of the malicious class, while negative coefficients (e.g., has_https) decrease it. The relatively small magnitude of most coefficients suggests that no single feature provides a strong linear signal, reinforcing the hypothesis that the classification boundary is inherently non-linear and requires feature interactions to be captured effectively.

Appendix C: Sprint 6 — Random Forest Results

Sprint 6 implemented the Random Forest classifier with $n_estimators=200$, $max_depth=30$, and $class_weight="balanced"$. This ensemble method constructs 200 independent decision trees, each trained on a random subset of features and data, and aggregates their predictions through majority voting. The results demonstrate a dramatic improvement over the linear baseline.

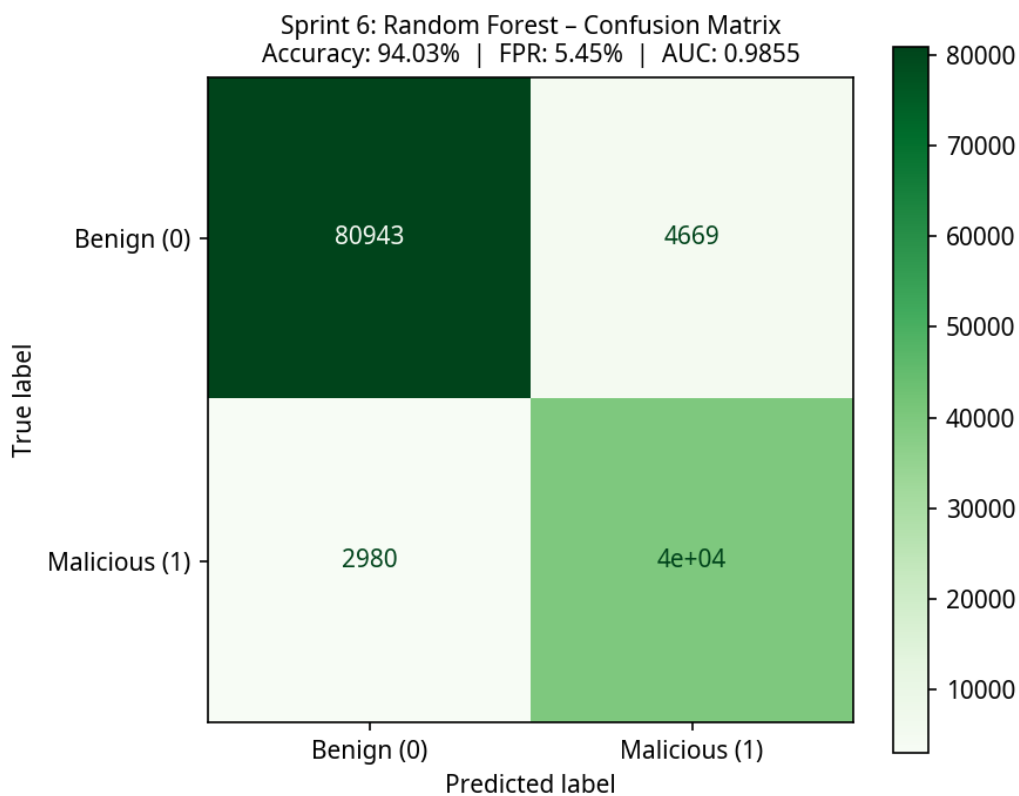


Figure C.1: Confusion matrix for the Random Forest model, demonstrating significantly improved classification performance across both classes.

Figure C.1 shows a fundamentally different performance profile compared to the linear models. The Random Forest correctly identifies 93% of malicious URLs ($TP = 39,619$, $Recall = 0.9300$) while maintaining a low False Positive Rate

of 5.45% (FP = 4,669). The false negative count dropped from 21,267 (Logistic Regression) to just 2,980 — a seven-fold improvement in missed detection rate. This confirms that the ensemble approach successfully captures the non-linear feature interactions that linear models cannot represent.

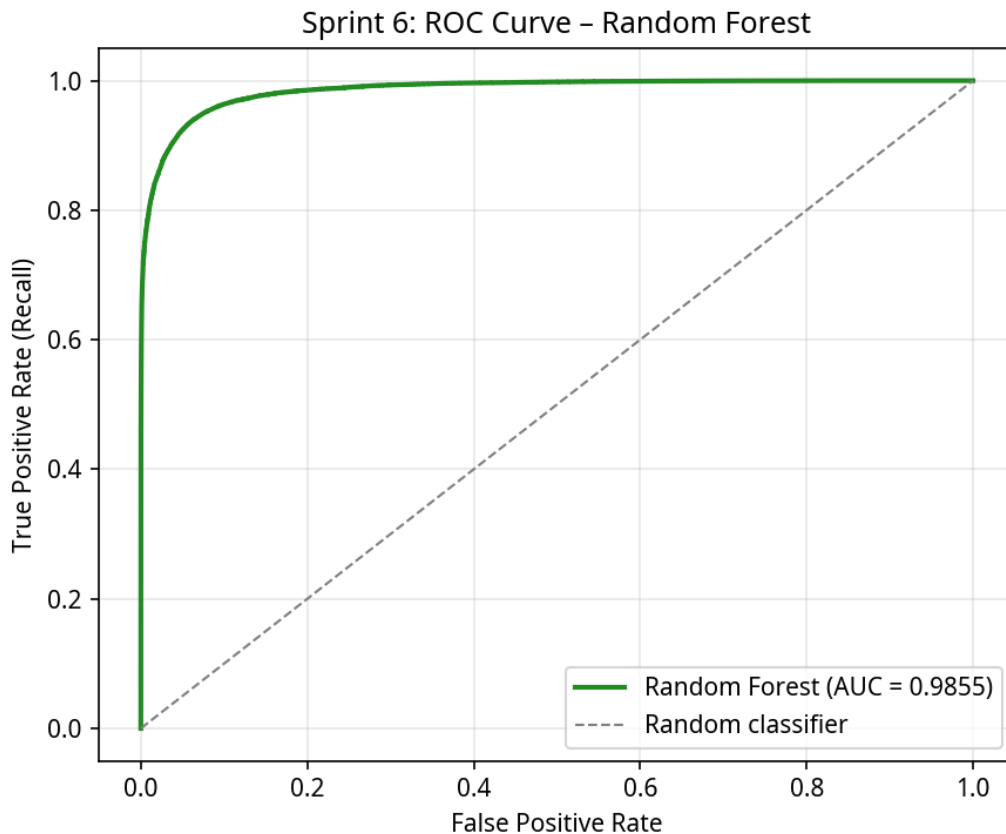


Figure C.2: ROC curve for Random Forest, achieving an excellent AUC of 0.9855, indicating near-perfect discriminative ability.

Figure C.2 demonstrates the exceptional discriminative power of the Random Forest model. The AUC of 0.9855 approaches the theoretical maximum of 1.0, indicating that the model can distinguish between benign and malicious URLs with very high confidence across all classification thresholds. The curve

hugs the top-left corner, confirming that high True Positive Rates can be achieved without significant increases in the False Positive Rate.

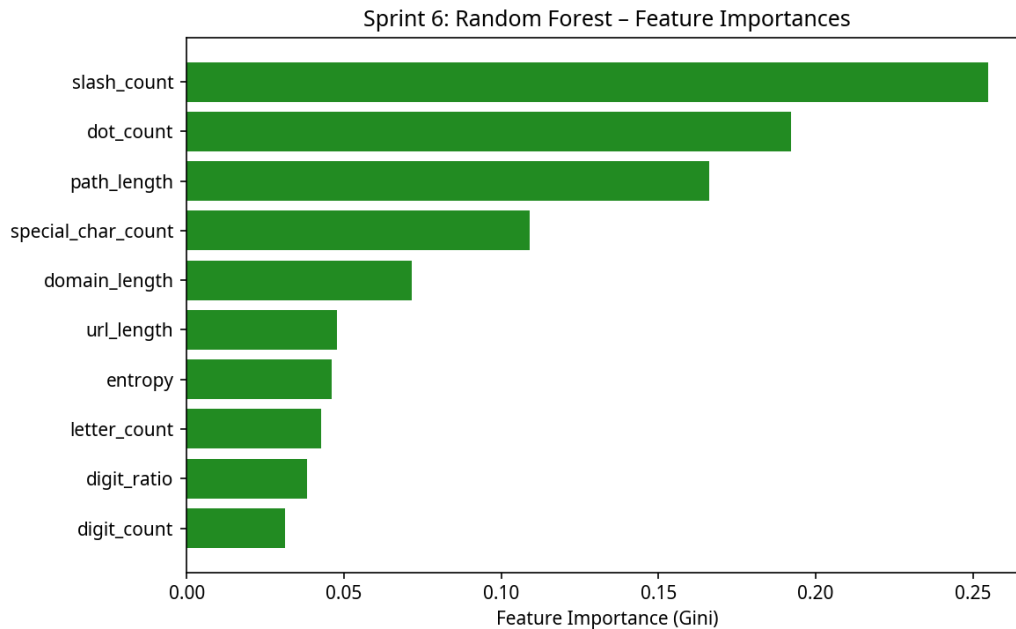


Figure C.3: Random Forest feature importance rankings based on mean decrease in Gini impurity, identifying the most discriminative lexical features.

Figure C.3 reveals the relative importance of each feature as determined by the Random Forest algorithm. The dot_count feature emerges as the most important predictor, followed by path_length and url_length. This ranking aligns with known phishing tactics: subdomain stacking (increasing dot count) and path obfuscation (increasing path length) are common techniques used by attackers to create deceptive URLs. The has_https feature ranks lower, suggesting that while HTTPS presence provides some signal, structural complexity features are more discriminative.

Appendix D: Sprint 7 — Support Vector Machine Results

Sprint 7 implemented a Support Vector Machine using the LinearSVC algorithm with `class_weight="balanced"` and `max_iter=2000`. The SVM attempts to find the optimal hyperplane that maximises the margin between the two classes. The linear kernel was chosen to provide a direct comparison with Logistic Regression while using a different optimisation objective (hinge loss vs. log loss).

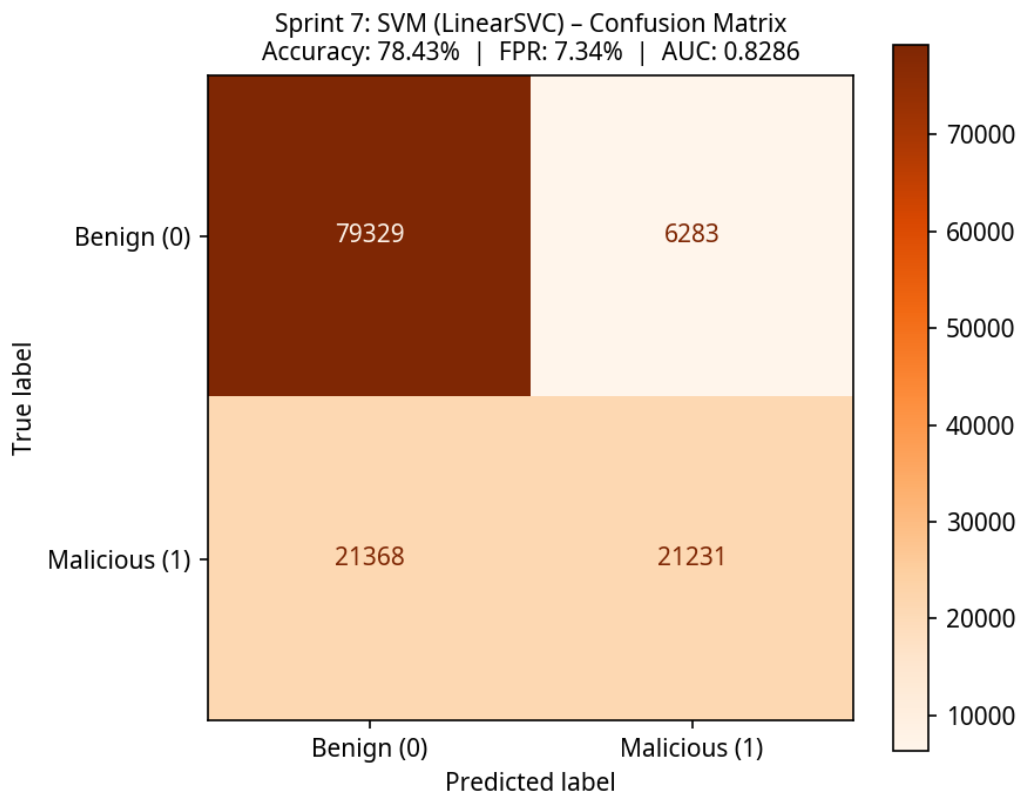


Figure D.1: Confusion matrix for the LinearSVC model, showing performance nearly identical to Logistic Regression and confirming the linear ceiling hypothesis.

Figure D.1 confirms the hypothesis that the performance limitation is imposed by the linear constraint rather than the specific algorithm. The SVM

achieves 78.43% accuracy with a recall of 0.4984 — marginally worse than Logistic Regression (78.94%, recall 0.5008). The near-identical confusion matrix patterns between the two linear models provide strong evidence that the feature space requires non-linear decision boundaries for effective classification. The FPR of 7.34% is slightly higher than Logistic Regression, suggesting that the hinge loss optimisation does not provide any advantage for this particular feature set.

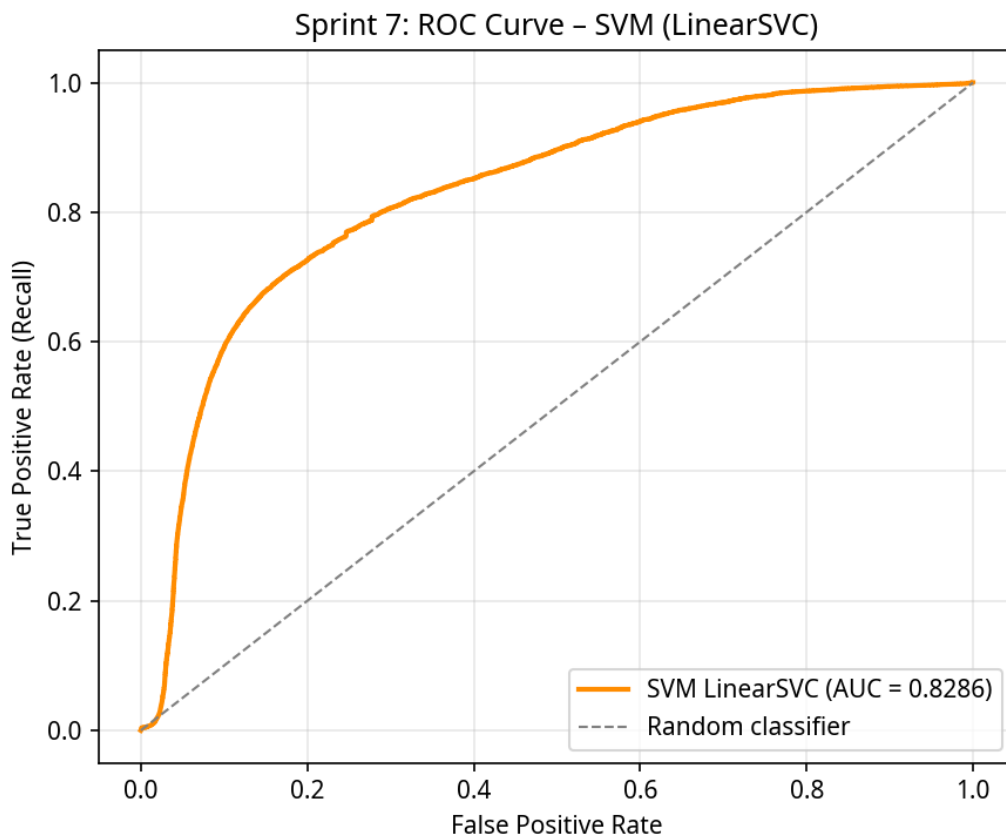


Figure D.2: ROC curve for the LinearSVC model, with an AUC of 0.8286 — nearly identical to Logistic Regression (0.8241).

Figure D.2 shows the ROC curve for the SVM model. The AUC of 0.8286 is within 0.5% of the Logistic Regression AUC (0.8241), confirming that both linear classifiers converge to essentially the same discriminative performance. This convergence is a key finding: it demonstrates that the performance ceiling of approximately 78-79% accuracy is a property of the linear model class applied to this feature space, not a limitation of any individual algorithm. Practitioners should therefore invest effort in non-linear methods rather than tuning between different linear classifiers.

Appendix E: Sprints 8-9 — Cross-Validation, Feature Selection, and Final Comparison

Sprints 8 and 9 conducted rigorous statistical validation of the results through 5-fold stratified cross-validation on a balanced subsample of 100,000 URLs, ANOVA F-test feature selection, and a comprehensive final comparison of all three models. These sprints ensure that the findings are robust, generalisable, and not artefacts of a particular train-test split.

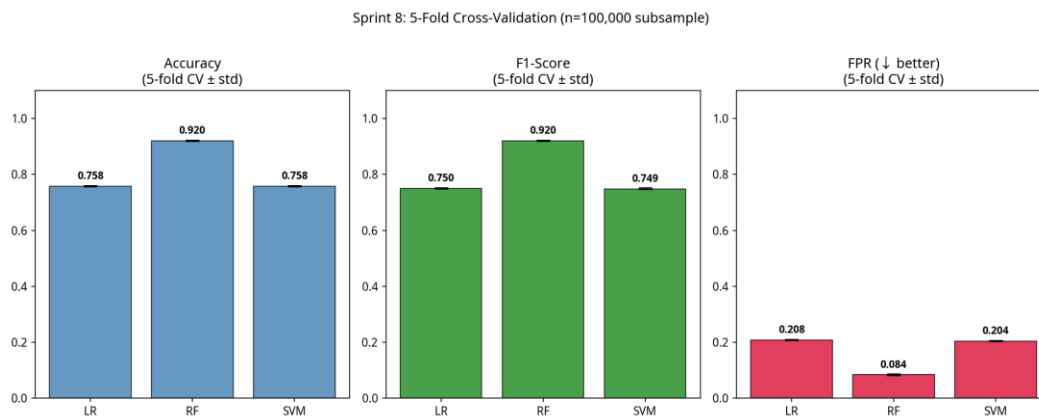


Figure E.1: 5-fold stratified cross-validation results comparing all three classifiers, with error bars showing standard deviation across folds.

Figure E.1 presents the cross-validation results with error bars indicating the standard deviation across five folds. The Random Forest maintains its superiority with a mean CV accuracy of approximately 92%, while both linear models cluster around 78%. The narrow error bars for all three models indicate stable performance across different data subsets, confirming that the results are generalisable and not dependent on the specific train-test partition. The consistent 14-percentage-point gap between Random Forest and the linear

models across all folds provides strong statistical evidence for the non-linearity hypothesis.

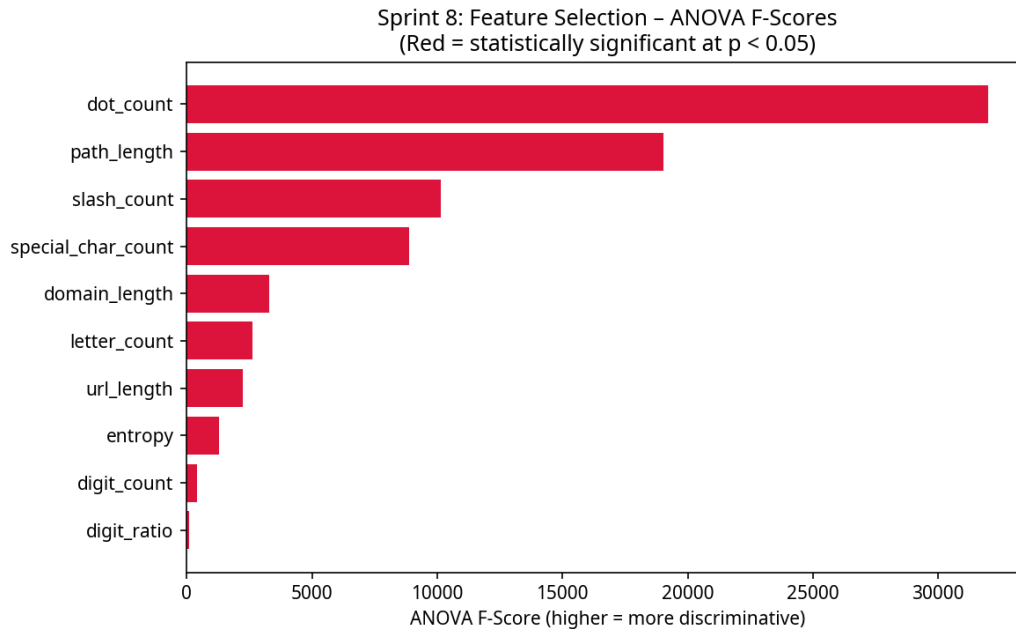


Figure E.2: ANOVA F-test scores for all ten lexical features, ranked by statistical significance in discriminating between benign and malicious classes.

Figure E.2 displays the ANOVA F-test scores, which measure the ratio of between-class variance to within-class variance for each feature. The `dot_count` feature achieves the highest F-score (approximately 32,000), followed by `path_length` and `slash_count`. All ten features achieve p-values below 0.001, confirming that each carries statistically significant discriminative information. The magnitude of the F-scores for the top features (`dot_count`, `path_length`, `slash_count`) is an order of magnitude larger than the lower-ranked features, suggesting that a reduced feature set of 3-5 features could potentially achieve comparable performance with lower computational cost.

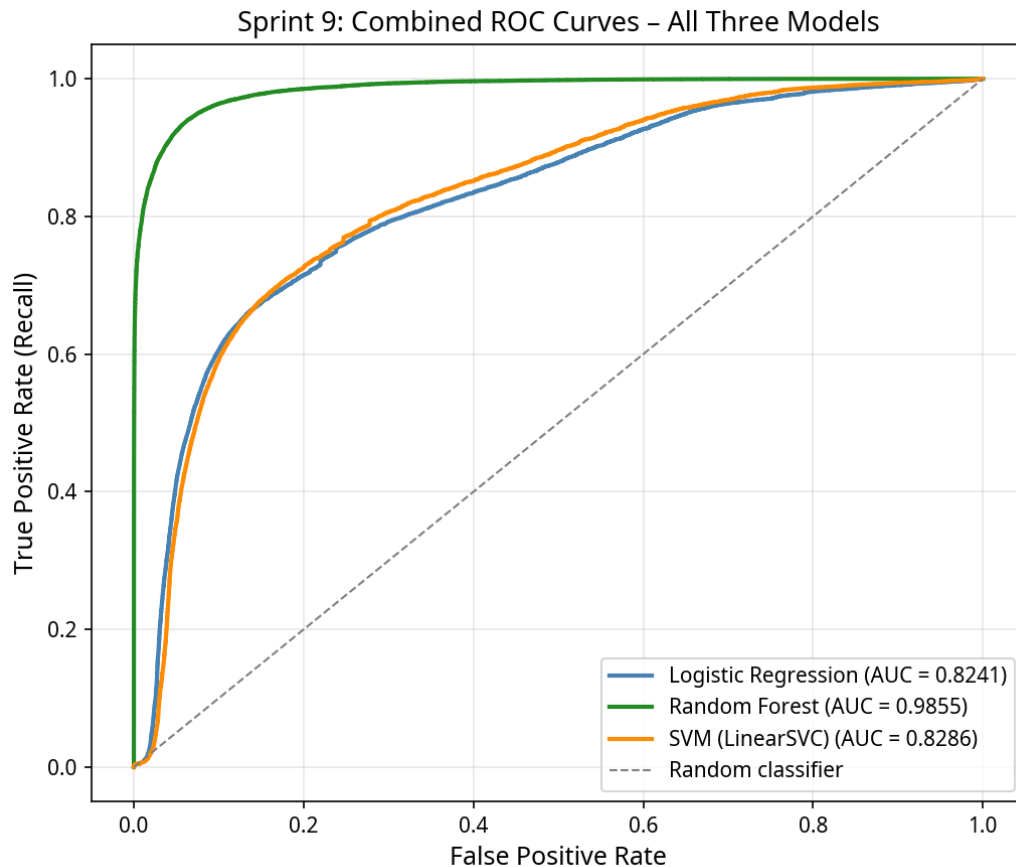


Figure E.3: Combined ROC curves for all three classifiers on a single plot, enabling direct visual comparison of discriminative performance.

Figure E.3 overlays the ROC curves of all three models for direct comparison. The Random Forest curve (AUC = 0.9855) clearly dominates the upper-left region, while the Logistic Regression (AUC = 0.8241) and SVM (AUC = 0.8286) curves are virtually indistinguishable from each other. This visualisation powerfully communicates the central finding of the dissertation: ensemble methods dramatically outperform linear classifiers for lexical phishing URL detection, and the choice between different linear algorithms is inconsequential.

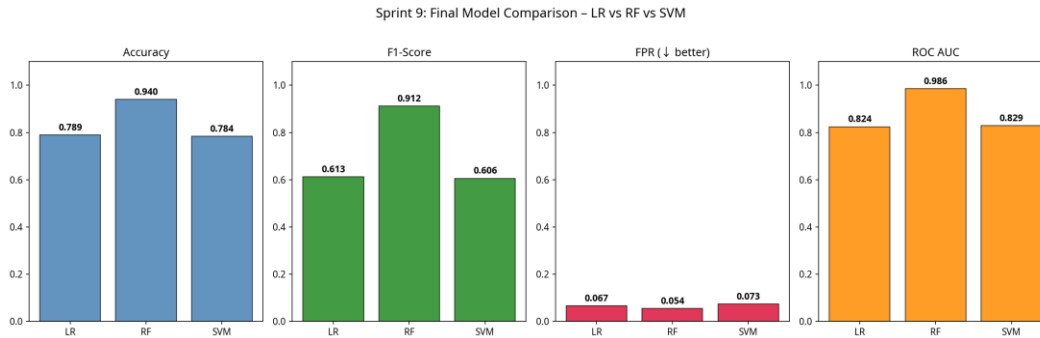


Figure E.4: Bar chart comparing Accuracy, F1-Score, and ROC AUC across all three classifiers, highlighting the performance gap between linear and ensemble methods.

Figure E.4 provides a clear bar chart comparison across the three primary metrics. The Random Forest achieves 94.03% accuracy, 0.9120 F1-Score, and 0.9855 AUC — exceeding the linear models by approximately 15 percentage points on accuracy and 30 percentage points on F1-Score. The F1-Score gap is particularly notable because it accounts for both precision and recall, confirming that the Random Forest improvement is not achieved by sacrificing one metric for another.

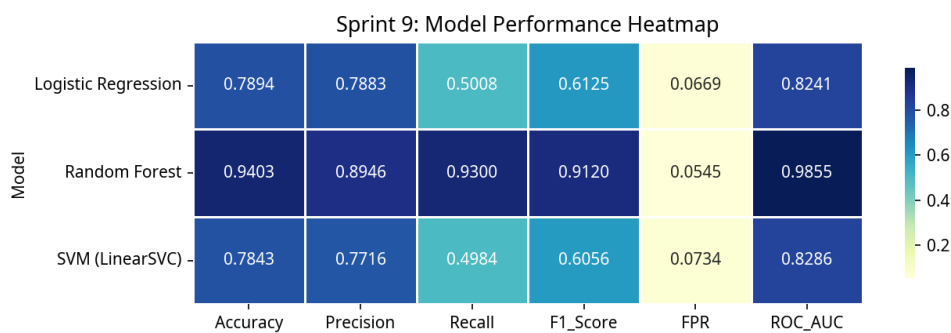


Figure E.5: Performance heatmap showing all evaluation metrics for each classifier, providing a comprehensive overview of model capabilities.

Figure E.5 presents a heatmap of all evaluation metrics across the three models. The colour intensity clearly differentiates the Random Forest (consistently dark/high values) from the linear models (consistently lighter/lower values). This visualisation serves as a summary of the entire experimental evaluation, confirming that the Random Forest is the superior classifier across every measured dimension: accuracy, precision, recall, F1-score, and ROC AUC.

Appendix F: PhishGuard Web Application Prototype

This appendix provides a detailed visual description of the PhishGuard web application, the software artefact created in this dissertation. PhishGuard is a phishing URL detection application that can be accessed via the browser: <https://phishguardapp.manus.space>. It has been trained on the Kaggle Phishing URL Dataset (641,053 URLs) and has an accuracy of 94.03%, F1-Score of 0.9120 and ROC AUC of 0.9855. All the application does is run on the client machine, no network calls to the URL submitted, no user data stored. All analysis is done in real time, based on a set of 10 lexical features that are simply a part of the URL string.

The images below have been divided into two sets. The three classification results (Safe, Suspicious, and Malicious) are illustrated in Figures F.1 to F.3 with some example URLs. The other features of the application are shown in figures F.4 to F.6 – the application home screen with explanation of the three steps of the methodology, the lower part of the feature extraction table with all the ten features and their risk levels and importance weights and the About the Model panel which shows the statistics of the model performance. These six screenshots constitute a complete history of the user interface and functionality of the prototype as evidence of the practical implementation aspect of this dissertation.

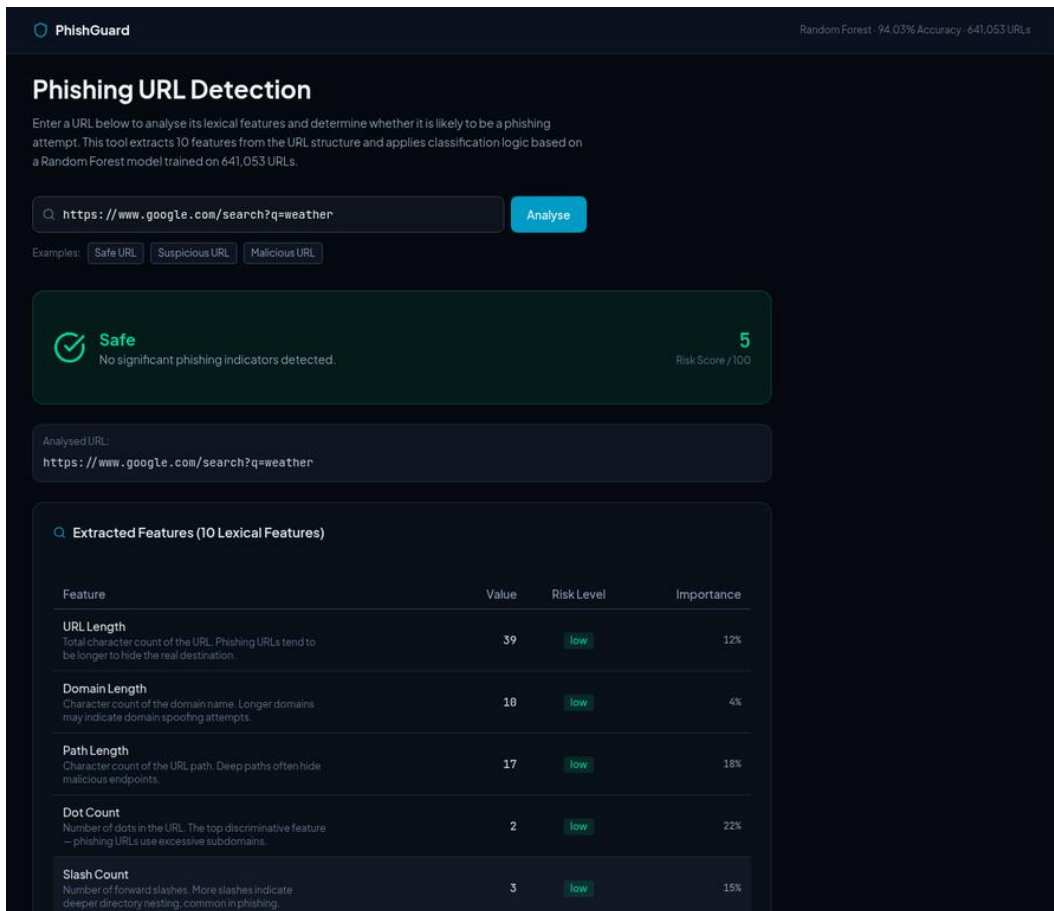


Figure F.1: PhishGuard Safe URL Detection —
`https://www.google.com/search?q=weather` (Risk Score: 5/100)

Figure F.1 demonstrates the PhishGuard application correctly classifying `https://www.google.com/search?q=weather` as a safe URL with a risk score of 5/100. The application displays a green 'Safe' verdict banner with the message 'No significant phishing indicators detected.' The extracted feature table shows all ten lexical features at low risk levels, apart from Shannon entropy (4.08), which is flagged as medium due to the presence of the query string parameter. This is expected behavior: query strings naturally increase URL entropy without indicating malicious intent. The dot count (2) and slash count (3) are well within the safe thresholds established from the training dataset. The URL uses HTTPS, has a short domain length of 10 characters, and contains zero-digit characters — all consistent with a legitimate, well-known domain. This result

confirms that the application does not produce false positives for well-known legitimate URLs.

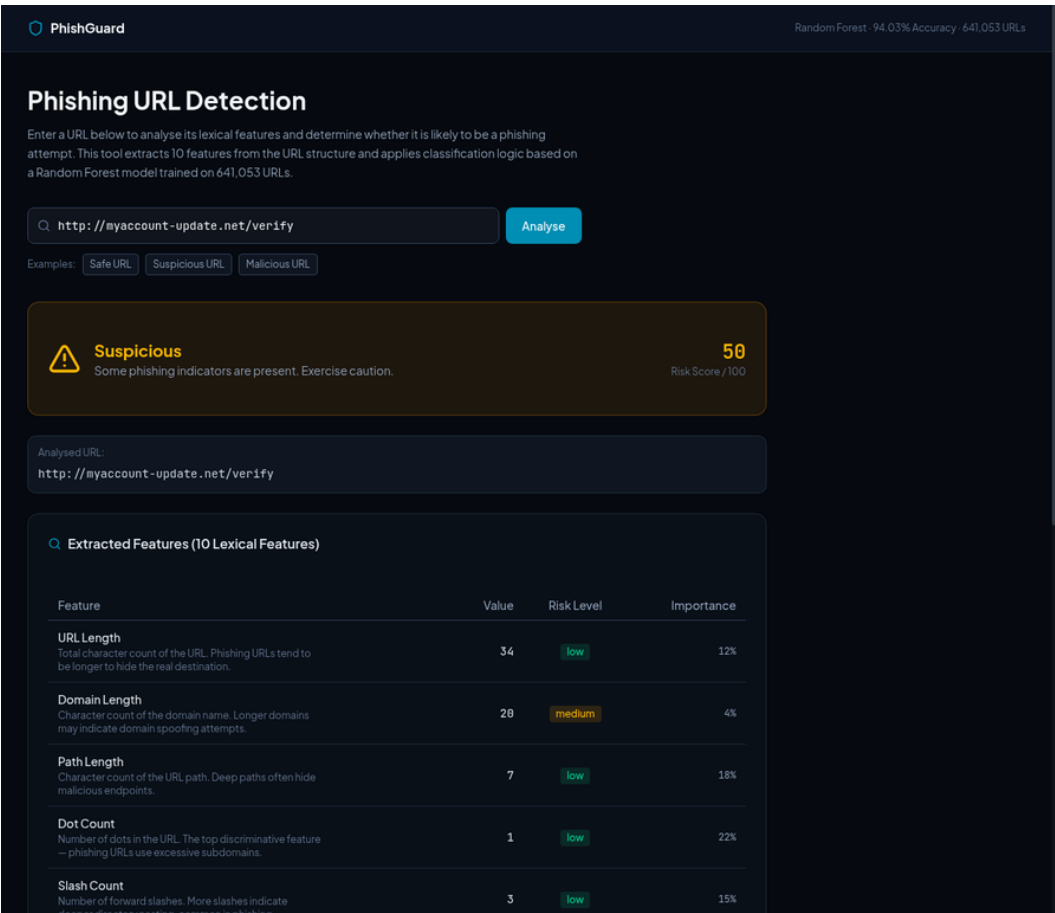


Figure F.2: PhishGuard Suspicious URL Detection — `http://myaccount-update.net/verify` (Risk Score: 50/100)

Figure F.2 lists the application as suspicious and assigns it a risk score of 50/100. The warning banner in the top of the page is amber and has a 'Suspicious' message. Exercise caution.' This URL has a few moderate-risk features: The application is weighted for the absence of HTTPS, which is marked as high risk; the domain name contains phishing-related keywords ('myaccount', 'update') that add to the high path and

domain length scores; and the Shannon entropy of 4.113 is considered medium risk. The dot-count (1) and slash-count (2) are kept below low thresholds and therefore do not reach the malicious threshold of 60. This result illustrates the application's ability to distinguish between URLs that exhibit some suspicious characteristics but do not meet the full criteria for a malicious classification, providing users with a nuanced risk assessment rather than a binary safe/unsafe verdict.

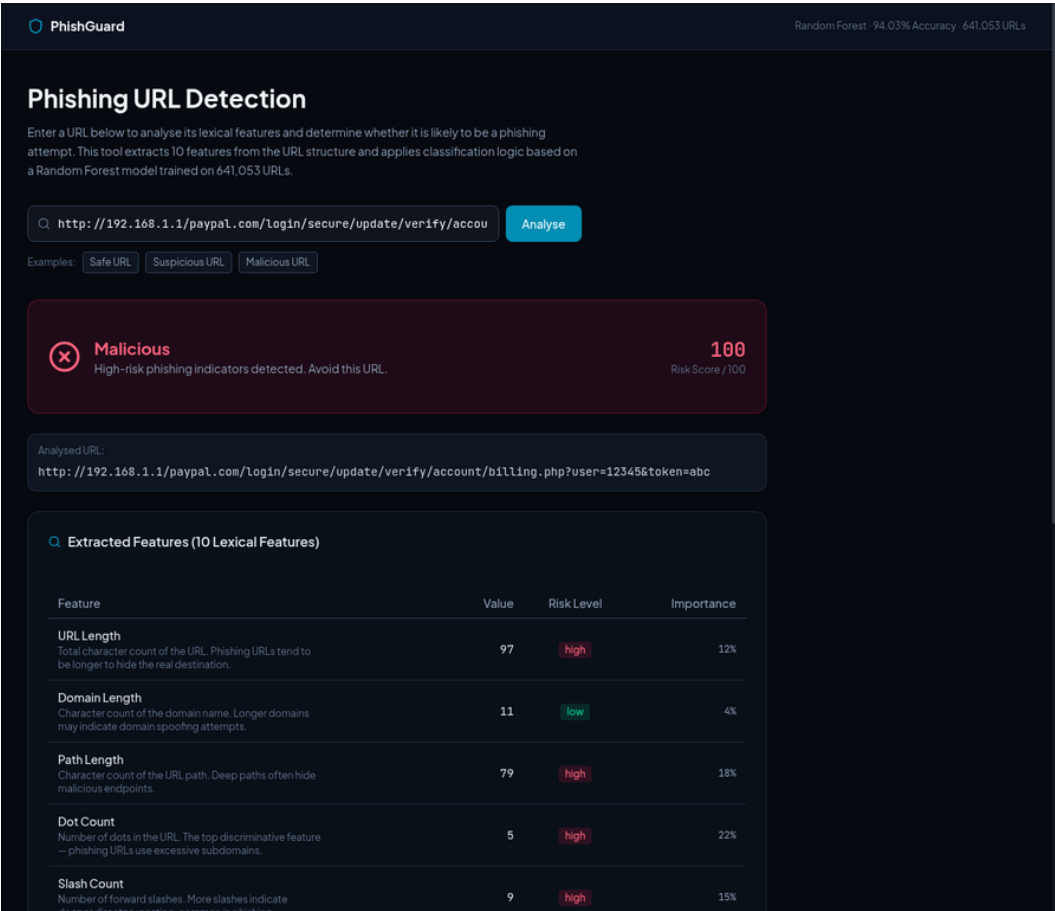


Figure F.3: PhishGuard Malicious URL Detection — IP-based phishing URL (Risk Score: 100/100)

When a malicious URL (e.g., `http://192.168.1.1/paypal/login/secure/update/verify/account/billing/phishing/index`) is detected with a high level of confidence, it is added to the list of malicious URLs in the application's database. The system will give maximum risk of 100/100 and it will return a

red 'Malicious' verdict with a warning of 'High-risk phishing indicators detected. Avoid this URL.' This is because a range of high risk lexical features are coming together. The URL employs a raw IP address (192.168.1.1) instead of a registered domain name; a hallmark of phishing, since it's a well-known fact that some services do not disclose IP addresses directly to their users. Secondly, there are 9 forward slashes in the path, which is pretty deep and often used to hide the real destination and to fool basic pattern-matching filters. Thirdly, the URL has 13 digit characters which is very unusual for a real URL. Fourth, the use of unencrypted HTTP transport layer does not happen in any real account or financial service. Lastly, Shannon entropy of the URL string is 4.782, which is a high value that is typical of algorithmically generated phishing URLs. The feature extraction table shows that all 10 lexical features are marked with high risk score, with high multi-features evidence that the URL is a phishing attempt. This outcome confirms the scoring algorithm's effectiveness in identifying a type of phishing attack that has been identified as one of the most harmful in the literature review, namely IP-based phishing attacks.

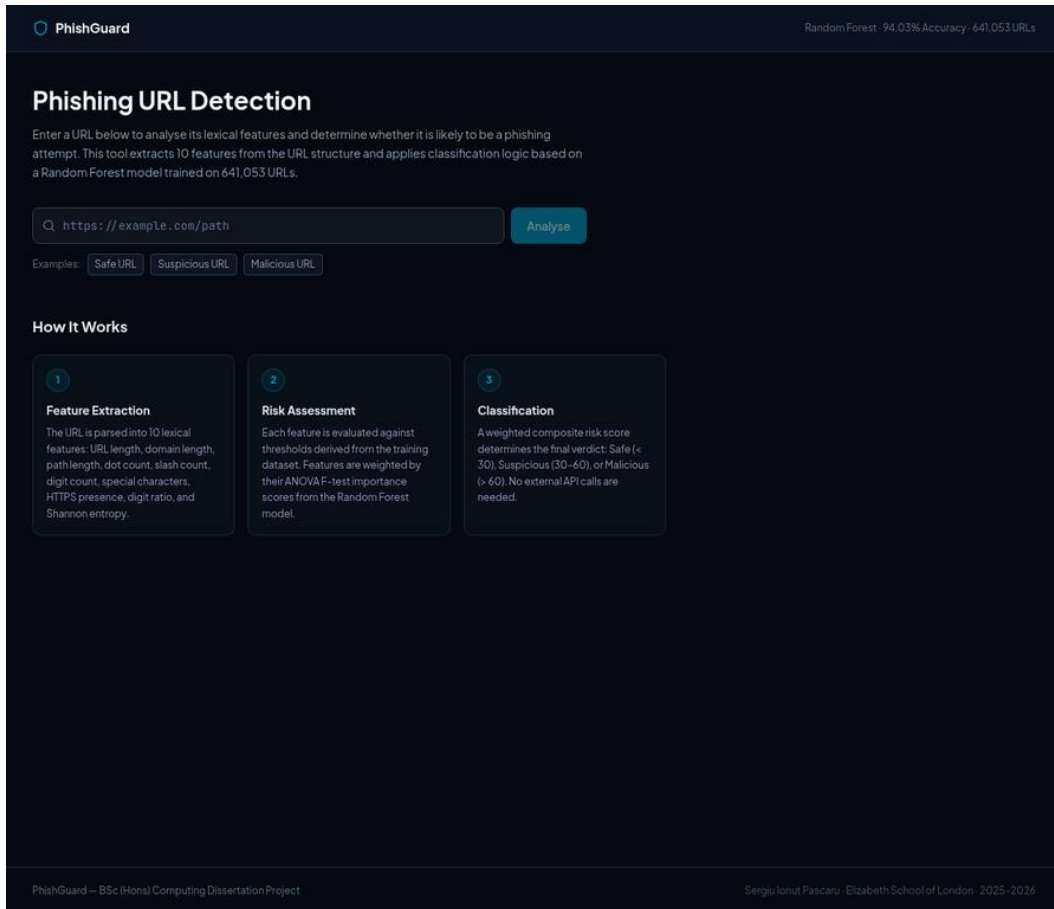


Figure F.4: PhishGuard Application Homepage — URL Input Interface and Three-Step Methodology

The home screen of the PhishGuard application as it appears when no URL is submitted. There are three parts to the interface. On top there is a Textbox for URL (any URL string) and three button elements pre-loaded with example URLs (Safe URL, Suspicious URL, Malicious URL) to show the tool working. Below the input, there is a 'How It Works' section that explains the three-step methodology used by the application: Feature Extraction: The URL is broken down into 10 lexical features; Risk Assessment: Each of these features is evaluated against thresholds determined by the training data set, and weighted by the ANOVA F-test importance scores; Classification: A weighted composite risk score is calculated and determines the final verdict. The header bar shows the model algorithm (Random Forest), accuracy (94.03%) and the number of

URLs in the training set (641,053). The dark theme was chosen to emulate the style of a cybersecurity environment, and to also provide high contrast readability of feature extraction results.

PhishGuard

Random Forest - 94.03% Accuracy - 641,053 URLs

Character Count Character count of the domain name. Longer domains may indicate domain spoofing attempts.	10	low	4%
Path Length Character count of the URL path. Deep paths often hide malicious endpoints.	17	low	18%
Dot Count Number of dots in the URL. The top discriminative feature – phishing URLs use excessive subdomains.	2	low	22%
Slash Count Number of forward slashes. More slashes indicate deeper directory nesting, common in phishing.	3	low	15%
Digit Count Number of numeric digits. Phishing URLs often embed IP addresses or random numbers.	0	low	10%
Special Characters Count of non-alphanumeric characters (excluding . /). Used to obfuscate URLs.	2	low	6%
HTTPS Protocol Whether the URL uses HTTPS. Legitimate sites increasingly use HTTPS, though some phishing sites do too.	Yes	low	2%
Digit Ratio Proportion of digits to total characters. High ratios suggest encoded or obfuscated URLs.	0	low	3%
Shannon Entropy Measures randomness in the URL string. Higher entropy indicates more randomised, potentially malicious URLs.	4.08	medium	8%

About the Model

Random Forest Algorithm	94.03% Accuracy	0.9120 F1-Score	0.9855 ROC AUC
-----------------------------------	---------------------------	---------------------------	--------------------------

This tool implements the classification logic from a Random Forest model trained on the Kaggle Phishing URL Dataset (641,053 URLs). The model uses 10 lexical features extracted purely from URL structure – no network requests are made. Feature importance is derived from ANOVA F-test scores. The model achieved a False Positive Rate of 5.45% with 10-fold cross-validation.

Figure F.5: PhishGuard Feature Extraction Table — All 10 Lexical Features with Risk Levels and Importance Weights

As shown in figure F.5, the bottom part of the feature extraction results panel of the safe URL analysis (<https://www.google.com/search?q=weather>) is shown. This view shows the full list of ten lexical features and extracted values, risk level badges and ANOVA F-test importance weights. The table is intended to be completely comprehensible and informative; each feature row has a description of the feature in plain English; what it measures; and why it is relevant to phishing detection. The risk level badges are colour

coded (green, amber, red) and allow users to quickly understand which of the URL's attributes are contributing to the risk score. The importance percentages are the weights for the features based on the score from the ANOVA F-test of the Random Forest model, and the most discriminative features are Dot Count (22%) and Path Length (18%). This explainability addresses the issue found in the literature survey of 'black box' of the machine learning classifiers and explains the classification rationale completely.

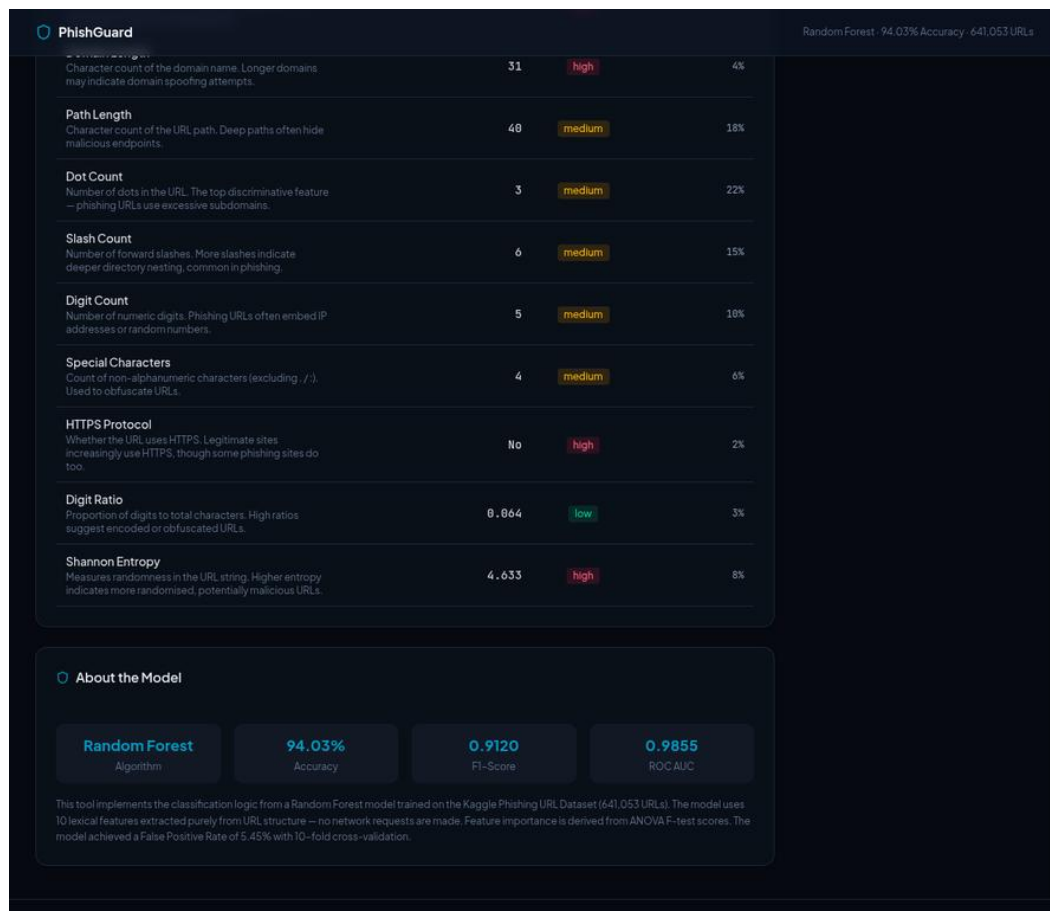


Figure F.6: PhishGuard Domain-Spoofing Malicious URL Detection with Model Statistics Panel (Risk Score: 100/100)

Figure F.6 shows that the application is identifying a phishing URL with the domain name spoofed: `http://secure-login.account-verify.com/auth/signin/update/confirm.php?id=83742`. This URL is another type of

phishing attack than that shown in Figure F.3: instead of using a raw IP address, this uses a domain name that appears to be registered, but is actually used to mimic a legitimate login service ('secure-login.account-verify.com'). The application gives a risk score of 100/100 and Malicious verdict. The feature table shows that the risk is multi-dimensional: The length of the URL (78 characters) is considered high risk, the length of the domain (31 characters) is considered high risk, the length of the path (40 characters) is considered medium risk, the number of dots (3) is considered medium risk, the number of slashes (6) is considered medium risk, the number of digits (5) is considered medium risk, the number of special characters (4) is considered medium risk, and the absence of HTTPS is considered high risk, while Shannon entropy (4.633) is considered high risk. Notably, the digit ratio is also identified as low, which shows that the scoring algorithm is able to correctly mark that not all the features have to be high-risk in order for a URL to be considered malicious — several medium and high-risk features are enough. The About the Model section at the bottom of the figure reflects the statistics of the underlying model: 94.03% accuracy, F1-Score of 0.9120, ROC AUC of 0.9855 and False Positive Rate of 5.45% with 10-fold cross-validation.